

SAPIENZA
UNIVERSITÀ DI ROMA

Seismic Viewer: Design, Implementation and Evaluation of an Interactive 4D Spatio-Temporal Visualisation System for Seismological Data

Faculty of Information Engineering, Computer Science and Statistics
Applied Computer Science and Artificial Intelligence

Alessio Curcio
ID number 2120317

Supervisor
Danilo Avola

Co-supervisor
Daniele Bailo

Academic Year 2025/2026

Seismic Viewer: Design, Implementation and Evaluation of an Interactive 4D Spatio-Temporal Visualisation System for Seismological Data
Bachelor's thesis. Sapienza University of Rome

© 2026 Alessio Curcio. All rights reserved

This thesis has been typeset by LaTeX and the Sapthesis class.

Author's email: alessio.curcio04@gmail.com

To my family, colleagues, and friends, by my side today or carved into my memories,
whose support is etched in the surface of every page,
shaping every step of this path.

Acknowledgments

First and foremost, I would like to express my gratitude to my academic supervisor, Prof. Danilo Avola, for his availability and for accepting to supervise my internship project. I am equally indebted to my internship supervisor, Dr. Daniele Bailo, for the trust he placed in me by welcoming me under his wing and for providing continuous guidance and mentorship during this project.

I am grateful to Giada and Rossella for their collaboration. Their constant readiness to share their knowledge and their moral support were a valuable help throughout this work.

A special thanks goes to Alessandro and all my friends from the "bunker". Thank you for being by my side through the many highs and lows of these past three years of my bachelor's degree. I also want to extend my sincere appreciation to Gianmaria for his patience and help during our countless hours of study together.

Finally, my heartfelt gratitude goes to my family for their patience, emotional support, and for always believing in me.

Abstract

This thesis documents the design, implementation, and evaluation of **Seismic Viewer**, a desktop application for four-dimensional (spatial and temporal) visualisation and analysis of seismological data. The system was developed during an internship to address a limitation in existing tools: the lack of software capable of rendering large hypocentral point clouds in real time while integrating spatial clustering and structural analysis.

Traditional seismic analysis often relies on two-dimensional maps or static plots that do not show the spatial and temporal evolution of earthquake sequences. Seismic Viewer addresses this limitation through a dual architecture that combines a Python computational backend (using vectorised numerical computing, clustering algorithms, and geospatial processing) with a hardware-accelerated WebGL frontend rendered through an embedded web engine.

The application supports four visualisation modes: 2D geospatial mapping, 3D static hypocentral distribution, 4D time-animated sequences, and interactive cluster analysis with structural plane fitting. It integrates data from local CSV files, the INGV GOSSIP online API, GNSS vertical ground deformation, and seismic tomography datasets. Analytical algorithms, including auto-tuned DBSCAN, MiniBatchKMeans clustering, RANSAC plane fitting, and a label matching method for temporal consistency, are used to identify geological structures in the data.

This thesis presents the development process from requirements and system architecture to implementation, testing, and performance optimization. The application was evaluated using earthquake data from the Campi Flegrei caldera, showing that it can render and analyze datasets with more than 50,000 seismic events in real time.

Contents

1 Introduction **1**
1.1 Motivation and Context . . . 1
1.2 Problem Statement . . . 1
1.3 Objectives and Scope . . . 2
1.4 Internship Context . . . 3
1.5 Document Structure . . . 3

2 Background and Related Work **5**
2.1 Seismological Fundamentals . . . 5
2.2 The Campi Flegrei Volcanic Complex . . . 6
2.3 Existing Seismic Visualisation Tools . . . 6
2.4 Machine Learning in Seismology . . . 8
2.5 Technology Choices and Justifications . . . 9
2.5.1 Architectural Trade-Off: pywebview vs. Electron . . . 9
2.5.2 Rendering Engine Selection: Plotly.js vs. Low-Level Graphics APIs . . . 10
2.5.3 Backend Server Framework: FastAPI vs. Traditional Frameworks . . . 10

3 System Architecture **11**
3.1 Architectural Overview . . . 11
3.2 Backend Architecture . . . 13
3.2.1 Application Entry Point & Bridge Engine . . . 13
3.2.2 Session Store & FastAPI Middleware . . . 13
3.3 Frontend Architecture . . . 14
3.3.1 Single-Page Application & UI/UX Design . . . 14
3.4 Data Flow Pipeline and Protocols . . . 14
3.4.1 Binary Data Transfer Protocol . . . 15
3.5 Threading and Concurrency Model . . . 16

4 Data Processing and Geospatial Engine **17**
4.1 CSV Ingestion and Schema Detection . . . 17
4.2 Data Sanitisation and Validation . . . 17
4.3 Online Data Import: The INGV GOSSIP API . . . 18
4.4 Geospatial Computations and Mesh Generation . . . 19
4.4.1 UTM Zone 33N to WGS84 Conversion . . . 19

4.4.2 Optimised Map Mesh and Caching Architecture 20
4.5 Pre-computation and Memory Layout Transformation 20

5 Visualisation Modes 21
5.1 2D Geospatial Map View 21
5.1.1 Map Layers and Real-Time Filtering 21
5.2 3D Static View 23
5.3 3D Animated View (4D Visualisation) 26
5.4 3D Cluster Analysis View 27
5.4.1 Cluster Visualisation and Interactive Tuning 27
5.4.2 Sub-Clustering 28
5.5 Additional Data Layers & View Caching 28
5.5.1 Auxiliary Layers: Tomography, Uplift, and Stations 28
5.5.2 View Caching and Navigation 31

6 Clustering and Structural Analysis Algorithms 32
6.1 Coordinate Projection and Spatial Clustering Methods 32
6.1.1 DBSCAN, MiniBatchKMeans, and Cylinder Partitioning . . 32
6.2 Automatic Parameter Tuning 33
6.3 Structural Plane Fitting and Geometric Synthesis 34
6.3.1 RANSAC Robust Plane Estimation 34
6.3.2 SVD Refinement and Strike/Dip Mechanics 35
6.4 Temporal Stability, Parallel Execution, and Complexity 35
6.4.1 Temporal Stability via Global Label Matching 36
6.4.2 Parallel Execution Architecture 36
6.4.3 Asymptotic Complexity Analysis 36

7 Development Process and Challenges 37
7.1 Development Timeline & Iterative Methodology 37
7.2 Key Technical Challenges and Solutions 37
7.2.1 Rendering Performance and UX Optimisation 38
7.2.2 Backend Architecture, Concurrency, and Algorithmic Resilience 39
7.3 Code Quality and Architecture 41
7.4 Testing, Verification, and Quality Assurance 41

8 Conclusions and Future Work 43
8.1 Summary of Contributions 43
8.2 Validation and Quantitative Performance Assessment 44
8.2.1 Experimental Benchmarks: Serialization and Streaming . . . 44
8.2.2 Clustering Latency and Algorithmic Throughput 45
8.2.3 Synthetic Validation of Structural Plane Extraction 45
8.3 Future Research and Architectural Extensions 45
8.4 Concluding Remarks 46

Supplementary Technical Details **47**
.1 Seismic Data Structures 47
.1.1 Input Catalog Format 47
.1.2 Internal Application State 48
.2 Network Protocol and Message Schemas 48
.2.1 WebSocket Message Specification 48
.2.2 Core Default Configuration Parameters 49

List of Figures

3.1 Dual-process hybrid architecture of Seismic Viewer with dedicated communication channels. 11
3.2 The Seismic Viewer single-page application structure, showing initial data loading (a) and the multi-mode analytical selection hub (b). . . 14
3.3 Linear data pipeline from input parsing to WebGL drawing. 15
3.4 Sequence of asynchronous execution and non-blocking streaming pipeline. 15
3.5 Concurrency hierarchy linking UI, server thread, and analysis worker threads. 16

4.1 The INGV GOSSIP API import dialog for configuring network criteria, temporal bounds, and quality thresholds. 18

5.1 The 2D Geospatial Map View in the Seismic Viewer. This workspace allows users to filter earthquake data (a) and combine it with advanced geophysical overlays, such as tomographic velocity maps (b) and ground uplift measurements (c). 23
5.2 The 3D Static View workspace. The user can explore the underground earthquake distribution (a) and combine it with 3D tomography models (b) and surface deformation data (c). 25
5.3 The 4D Animated Spatio-Temporal View workflow. Users configure parameters via the dialog (a), explore time-evolving seismicity (b), and observe automated fault plane tracking (c). 27
5.4 Multi-scale static (a) and animated (b to d) 3D/4D Cluster Analysis modes (Part 1). 29
5.4 Multi-scale static (a) and animated (b to d) 3D/4D Cluster Analysis modes (Part 2). 30

7.1 Development milestones highlighting the iterative implementation methodology. 37

List of Tables

2.1 Comparison of existing seismic visualisation tools against Seismic Viewer. 8
2.2 Summary of the selected technology stack and rejected alternatives. 9

3.1 REST API endpoints and WebSocket message protocol. 12
3.2 Binary buffer layout (Float32). 16

6.1 Comparison of spatial clustering approaches in Seismic Viewer. . . . 33
6.2 Asymptotic time and space complexity of the analytical methods. . . 36

8.1 Serialization benchmark: JSON vs. Float32 Binary buffer. 44
8.2 Clustering execution latency benchmark across dataset scales. 45
8.3 Synthetic plane estimation accuracy under Gaussian positional noise. 45

.4 Expected schema for input seismic catalogs. 47
.5 Internal data representation in the processing pipeline. 48
.6 Centralized configuration constants and default thresholds (`core/config.py`). 49

Chapter 1

Introduction

1.1 Motivation and Context

Seismology is a data-intensive domain within Earth sciences. Seismic monitoring networks continuously record events, generating datasets that describe subsurface processes such as tectonic stress accumulation and magmatic fluid migration. Interpreting hypocentral data in near-real time is important for volcanic and seismic risk assessment.

Traditionally, earthquake catalogues have been analyzed through two-dimensional maps, cross-sections, and time-series plots. These representations do not fully capture the four-dimensional spatial and temporal distribution of seismic processes. An earthquake sequence migrating along a fault at depth may appear as a diffuse group in map view, but reveal a planar structure when examined in three dimensions and animated over time. This limitation of conventional visualisation tools motivated the development of **Seismic Viewer**.

The project was developed during an internship at the Istituto Nazionale di Geofisica e Vulcanologia (INGV). Researchers studying the Campi Flegrei caldera required a tool to display spatial distributions, temporal evolution, and structural relationships within seismic sequences in an interactive desktop application.

1.2 Problem Statement

Despite the availability of numerous Geographic Information System (GIS) platforms and seismological software packages, a significant technological gap persists. Existing tools generally fall into one of two categories:

(a) **General-purpose GIS platforms** (e.g., QGIS, ArcGIS): These offer geospatial analysis capabilities but are not designed for rendering dynamic 3D point clouds with seismic metadata in real time. Their architectures prioritize cartographic precision over interactivity, leading to delays when handling datasets with tens of thousands of hypocentres. From a software architecture perspective, traditional GIS tools often use synchronous data processing and standard text formats like JSON. When handling large datasets with tens of thousands of points, this causes the user interface to freeze due to high memory usage and parsing delays.

(b) **Specialised seismological software** (e.g., Seisan, GMT, ObsPy): These provide robust analytical functionality but typically produce static outputs, such as static figures for print rather than interactive explorations. The learning curve is steep, requiring command-line proficiency, and the visualisation paradigm remains fundamentally two-dimensional.

Neither category adequately addresses the need for a tool that unifies *real-time 3D rendering*, *temporal animation*, *machine learning-based clustering*, and *structural geological analysis* within a single, accessible desktop application. Seismic Viewer was designed to fill this gap.

1.3 Objectives and Scope

The primary objectives of this project are:

1. **Interactive 4D Visualisation**: Develop a system capable of rendering seismic hypocentres in three spatial dimensions plus time, with smooth animations and real-time filtering.
2. **Automated Structural Analysis**: Integrate spatial clustering algorithms (DBSCAN, K-Means) and plane fitting (RANSAC + SVD) to automatically identify seismogenic structures. In particular, solve the cluster label switching issue across consecutive 4D animation frames to keep colors consistent over time.
3. **Multi-Source Data Integration**: Support ingestion from local CSV files, the INGV GOSSIP online API, seismic tomography datasets (Vp/Vs ratios), and GNSS ground deformation data.
4. **Performance at Scale**: Ensure sub-second interactive response times for large datasets, through a custom Float32 binary streaming protocol, an asynchronous multi-channel communication design (native bridge, HTTP REST, WebSockets), GPU-accelerated rendering, and efficient caching strategies.
5. **Accessible User Experience**: Provide a modern, intuitive interface that does not require programming expertise, enabling geophysicists to focus on interpretation rather than tool operation.

This report covers the development process of the application, from the initial design to testing and evaluation. The system was evaluated using earthquake data from the Campi Flegrei caldera. In addition to practical usability, this thesis explains the software engineering solutions used to connect Python scientific calculations with interactive 3D web graphics. The following chapters present the domain background, the system architecture, the data processing pipeline, the visualisation modes, the algorithmic foundations, the development process, and the conclusions drawn from this work.

1.4 Internship Context

This work was conducted as part of a university internship. The project was developed collaboratively with Giada Alfonsi, a PhD student in geology at Roma Tre University, under the supervision of Daniele Bailo, a software engineer at the INGV.

An iterative development approach was adopted, progressively expanding the software's capabilities over the duration of the internship. Rather than following a rigid specification, each development cycle was shaped by the continuous feedback and domain expertise provided by Giada Alfonsi, Professor Guido Giordano (volcanologist and professor at Roma Tre University), researchers at the INGV Osservatorio Vesuviano, and Daniele Bailo. The primary phases of the project were as follows:

- **Phase 1**: Initial prototype, basic 3D visualisation, and local CSV ingestion.
- **Phase 2**: Major architectural refactoring with the introduction of K-Means clustering, a GUI overhaul, online data import from INGV, and structural plane visualisation.
- **Phase 3**: Performance optimisation (binary streaming, cylinder-based clustering) and initial project documentation.
- **Phase 4**: Implementation of sub-clustering and auto-tuning of DBSCAN parameters via K-distance analysis.
- **Phase 5**: Integration of tomography and GNSS uplift layers, followed by the final modular refactoring of the codebase.

This collaborative and iterative feedback process proved essential for aligning the software with real analytical workflows, as several key features (such as the cylinder grid partitioning and the sub-clustering interface) emerged directly from practical requirements and discussions during development.

1.5 Document Structure

The remainder of this thesis is organised as follows:

Chapter 2 reviews the seismological and technological background, surveys related work, and justifies the key technology choices.

Chapter 3 presents the system architecture, detailing the backend-frontend separation, communication protocols, and threading model.

Chapter 4 describes the data processing pipeline, including CSV ingestion, online import, geospatial computations, and performance optimisations.

Chapter 5 documents the four visualisation modes and the integration of auxiliary data layers (tomography, uplift, station positions).

Chapter 6 formalises the clustering and structural analysis algorithms, with mathematical derivations and implementation details.

Chapter 7 reflects on the development process, discusses key technical challenges, and presents the solutions adopted.

Chapter 8 summarises the contributions, acknowledges limitations, and outlines directions for future work.

Chapter 2

Background and Related Work

This chapter outlines the theoretical and technical foundations of the project. It starts by introducing fundamental seismological concepts and presenting the Campi Flegrei volcanic complex, which serves as the primary case study for the application. Following a review of the current state of seismic visualisation tools, the chapter concludes by explaining and justifying the software tools selected for the development of Seismic Viewer.

2.1 Seismological Fundamentals

This section introduces the basic seismological concepts needed to understand the data and analysis in Seismic Viewer.

Hypocentres and Epicentres. When an earthquake occurs, energy is released underground along a fracture. The exact point inside the Earth where the rupture begins is called the **hypocentre** (or focus). It is defined by three coordinates: latitude (ϕ), longitude (λ), and depth (z). The point directly above the hypocentre on the surface of the Earth is the **epicentre**. Traditional maps often show only the epicentre on a flat surface, losing the depth information. Seismic Viewer preserves the full three-dimensional position of each hypocentre to show the true underground structure.

Magnitude Scales. The magnitude measures the energy released by an earthquake. In local monitoring catalogues, such as those for the Campanian volcanic area, the INGV primarily uses the **duration magnitude** (M_D). Seismic Viewer can automatically detect and load different magnitude types (M_D, M_L, M_W) during data import (see Section 4.1).

Fault Geometry: Strike, Dip, and Dip Direction. Earthquakes often occur along underground fracture surfaces called faults, which can be represented geometrically as tilted planes. To describe the orientation of a fault plane in three dimensions, geologists use three fundamental angles:

The **strike** (ϕ_s) is the compass direction of the fault line. It is defined as the orientation of a horizontal line lying on the tilted plane, measured in degrees

clockwise from North ($0° \leq \phi_s < 360°$). By convention, when looking along the strike direction, the fault plane slopes downward to the right.

The **dip** (δ) is the steepness or inclination of the fault plane. It is the angle between the tilted plane and a horizontal surface, measured along the line of maximum slope perpendicular to the strike. The dip angle ranges from 0° for a completely horizontal plane to 90° for a vertical fault.

The **dip direction** is the compass direction toward which the fault plane slopes downward into the subsurface. It is always perpendicular to the strike (at a 90-degree angle to the right):

$$\text{Dip Direction} = (\phi_s + 90°) \bmod 360° \tag{2.1}$$

Together, strike and dip define the complete 3D position of a geological fault. In Seismic Viewer, these angles are calculated automatically from earthquake clusters by fitting planes using the RANSAC [4] and SVD [17] algorithms (described in Chapter 6).

2.2 The Campi Flegrei Volcanic Complex

The Campi Flegrei caldera, situated west of Naples in southern Italy, is an active volcanic area in Europe.

Since 2005, Campi Flegrei has experienced ground uplift (*bradyseism*) accompanied by seismic swarms [2]. The INGV Osservatorio Vesuviano monitors the area through seismic stations, GNSS receivers, and geochemical sensors. The resulting datasets, comprising thousands of located earthquakes, daily GNSS displacement time series, and tomographic inversions, form the primary validation domain for Seismic Viewer.

Analyzing subsurface processes at Campi Flegrei, including fluid movement and fault activation, requires integrating multiple data sources within a shared spatial and temporal framework. This requirement motivated the multi-layer architecture of Seismic Viewer (see Section 5.5). Recent studies [5] also showed that earthquake hypocentres in this caldera can be approximated by planar surfaces, supporting the integration of plane fitting methods (RANSAC and SVD) into the application.

2.3 Existing Seismic Visualisation Tools

A variety of software tools are used in seismology, ranging from general Geographic Information Systems (GIS) to specialized geophysical software. However, existing solutions exhibit trade-offs between domain-specific functionality, interactivity, and ease of use.

QGIS and ArcGIS. These general-purpose GIS platforms excel at 2D cartography, spatial overlay analysis, and integrating seismic events with surface infrastructure and geological maps. However, their core architecture is designed for 2D raster and vector layers rather than dense, dynamic 3D point clouds. Although 3D views can be rendered natively or via plugins like Qgis2threejs, performance drops significantly

when handling tens of thousands of hypocentres. In addition, they lack built-in tools for interactive 4D time-windowed playback and automated structural fault plane fitting.

GMT and PyGMT. The Generic Mapping Tools (GMT) and its Python interface (PyGMT) represent the standard choice for producing high-resolution static maps, cross-sections, and focal mechanism plots. However, GMT is fundamentally a batch-processing, script-driven tool that generates static image files (such as PostScript or PNG). It does not provide an interactive graphical environment where an analyst can rotate 3D scenes in real time, interactively adjust clustering parameters, or observe dynamic earthquake migrations.

ObsPy. ObsPy is the standard open-source Python framework for seismological research, offering comprehensive tools for waveform processing, event metadata management, and standard file format support (such as MiniSEED, SAC, and QuakeML). While ObsPy is widely used for data ingestion and signal processing, it is a programming library rather than an interactive desktop application. Its visualisation capabilities rely mainly on static 2D Matplotlib figures.

Seisan. Seisan is a widely used software suite for earthquake observatory workflows, including phase picking, hypocentre location, and seismic database management. While reliable for routine monitoring, its user interface and codebase are based on legacy terminal utilities and Fortran routines, presenting a steep learning curve and lacking modern, hardware-accelerated 3D/4D visual exploration capabilities.

ParaView. ParaView is an open-source scientific visualisation engine based on VTK, capable of rendering large 3D and 4D volumetric datasets, such as seismic tomography models and wave propagation simulations. However, ParaView is a general-purpose scientific visualisation tool: it requires complex data pipeline configurations and lacks domain-specific seismological features, such as direct online catalog ingestion (e.g., the INGV GOSSIP API), automated hypocentral clustering, and strike/dip fault plane extraction.

GPlates. GPlates is an interactive plate tectonics tool designed for reconstructing geological continental motions across global scales and deep geological time (millions of years). While it supports 3D spheres and time animation, it is designed for global plate kinematic models rather than high-resolution, local earthquake sequences or real-time volcano-seismic monitoring.

Table 2.1 summarises the capabilities of these established tools and highlights the functional gap addressed by Seismic Viewer.

The comparison confirms that no existing software combines interactive 3D rendering, temporal 4D animation, automated machine learning clustering, real-time parameter tuning, and structural fault plane fitting within an accessible desktop application. Seismic Viewer was designed specifically to unify these capabilities.

Table 2.1. Comparison of existing seismic visualisation tools against Seismic Viewer.

Tool	3D View	4D Anim.	Cluster	Real-time	GUI	Primary Limitations
QGIS	Limited	×	Plugin	×	✓	General GIS, not optimised for dynamic 3D point clouds
GMT / PyGMT	✓	×	×	×	×	Static script-based outputs, no real-time interactivity
ObsPy	Limited	×	External	×	×	Library framework, no interactive desktop GUI
Seisan	Limited	×	×	×	Partial	Legacy observatory suite, steep learning curve
ParaView	✓	✓	×	Partial	✓	General-purpose VTK tool, lacks seismology workflows
GPlates	✓	✓	×	Partial	✓	Global plate tectonic focus, not local earthquake swarms
Seismic Viewer	✓	✓	✓	✓	✓	Dedicated 4D seismotectonic analysis in a standalone app

2.4 Machine Learning in Seismology

The application of machine learning to seismological data has grown substantially in recent years, building on established pattern recognition and clustering foundations [8] and supported by libraries such as scikit-learn [12]. Two families of algorithms are particularly relevant to the spatial analysis of hypocentral distributions, and Seismic Viewer integrates both to offer complementary analytical perspectives.

Density-Based Clustering: DBSCAN. The Density-Based Spatial Clustering of Applications with Noise (DBSCAN) algorithm [3] identifies clusters as regions of high point density separated by regions of low density. Unlike partitioning methods, DBSCAN does not require specifying the number of clusters *a priori* and can identify structures of arbitrary shape, which is useful when analysing non-planar fault zones. The algorithm uses two parameters: ε (the neighbourhood radius) and `MinSamples` (the minimum number of points required to form a dense region). Points outside these clusters are marked as **noise**, filtering out background seismicity to isolate structural features. Implementation details and parameter tuning strategies are discussed in Chapter 6.

Centroid-Based Clustering: K-Means. The K-Means algorithm [9] partitions a set of N observations into K clusters by minimising the within-cluster sum of squared distances to each cluster centroid. Seismic Viewer uses the **MiniBatchKMeans** variant, which processes random subsets (mini-batches) of data at each step. This reduces computation time from $O(NKT)$ to near-linear complexity, enabling fast reclustering as the user adjusts parameters. K-Means is suitable when the number of target structures is known or when spherical partitions are desired.

Temporal Tracking: Cluster Label Switching. When clustering point clouds

over time, unsupervised algorithms run independently on each animation frame and may assign different labels (and colors) to the same physical structure across successive time steps. In an animation, this causes colors to change between frames. To preserve consistent colors across 4D sequences, the system uses a label matching algorithm, described in Section 6.4.

2.5 Technology Choices and Justifications

To choose the technology stack, various alternatives were systematically compared, focusing on a balance between high performance, rapid development, and future maintainability. Table 2.2 summarises the key decisions.

Table 2.2. Summary of the selected technology stack and rejected alternatives.

Component	Chosen	Rejected	Justification
Language	Python 3.10+	Java, C++	Rich ecosystem for data science, rapid prototyping
Desktop Framework	pywebview [18]	Electron [10], Qt	Lightweight memory footprint, native file dialogs, direct Python to JS bridge
HTTP Server	FastAPI [14]	Flask, Django	Native async support, WebSockets, automatic API documentation, high throughput
Rendering	Plotly.js [13]	D3.js, Three.js, CesiumJS	High-level 3D API, native WebGL acceleration, built-in interactivity
Clustering	scikit-learn [12]	Custom, R	Reliable and highly optimized implementations, includes MiniBatchKMeans and DBSCAN
Maps	Smopy [15] (OSM [11])	Mapbox, Google Maps	Free and open-source, requires no API key, supports offline caching

2.5.1 Architectural Trade-Off: pywebview vs. Electron

Selecting the desktop framework affects memory usage and execution speed. While **Electron** [10] is widely used for desktop web apps, it bundles a Chromium browser and Node.js with the application. This configuration uses 150–250 MB of RAM in idle and requires inter-process communication between Node.js and Python.

In contrast, **pywebview** [18] uses the web component present in the operating system (WebView2 on Windows, WebKit on macOS). This keeps idle memory usage under 40 MB and avoids bundling an extra browser runtime. In addition, pywebview

provides a direct JavaScript–Python bridge (`js_api`), allowing UI actions and native file dialogs to execute without network overhead.

2.5.2 Rendering Engine Selection: Plotly.js vs. Low-Level Graphics APIs

Rendering 3D earthquake datasets interactively requires GPU acceleration via WebGL. Low-level graphics engines like **Three.js** provide control over visual elements, but require implementing 3D axes, projections, legends, and camera controls from scratch. Conversely, 2D charting tools like **D3.js** lack 3D WebGL support.

Plotly.js [13] combines WebGL acceleration with built-in scientific plotting features and interactive controls. By using Plotly's WebGL pipeline and preserving the camera state during updates (`uirevision`), Seismic Viewer achieves 60 FPS rendering for over 50,000 earthquakes without requiring a custom graphics engine.

2.5.3 Backend Server Framework: FastAPI vs. Traditional Frameworks

The backend streams binary data buffers and handles WebSocket messages (progress updates, parameter filters) alongside mathematical calculations. Synchronous Python web frameworks like **Flask** or **Django** can block incoming requests during long calculations unless coupled with external task queues.

FastAPI [14], running on the `uvicorn` [1] server, uses an asynchronous event loop via `asyncio`. This design supports WebSocket communication and HTTP endpoints within a single process. Numerical calculations are offloaded to background threads or process pools using `asyncio.run_coroutine_threadsafe`, keeping the user interface responsive.

Chapter 3

System Architecture

This chapter describes the system architecture of Seismic Viewer, detailing its structural decomposition, communication mechanisms, and execution model. The design balances two competing demands: the computational throughput required for scientific processing and the responsiveness expected of an interactive desktop application.

3.1 Architectural Overview

Seismic Viewer follows a **dual-process hybrid architecture** combining a Python scientific processing backend with an embedded Chromium-based frontend. The layers communicate via three distinct channels dedicated to specific operational tasks.

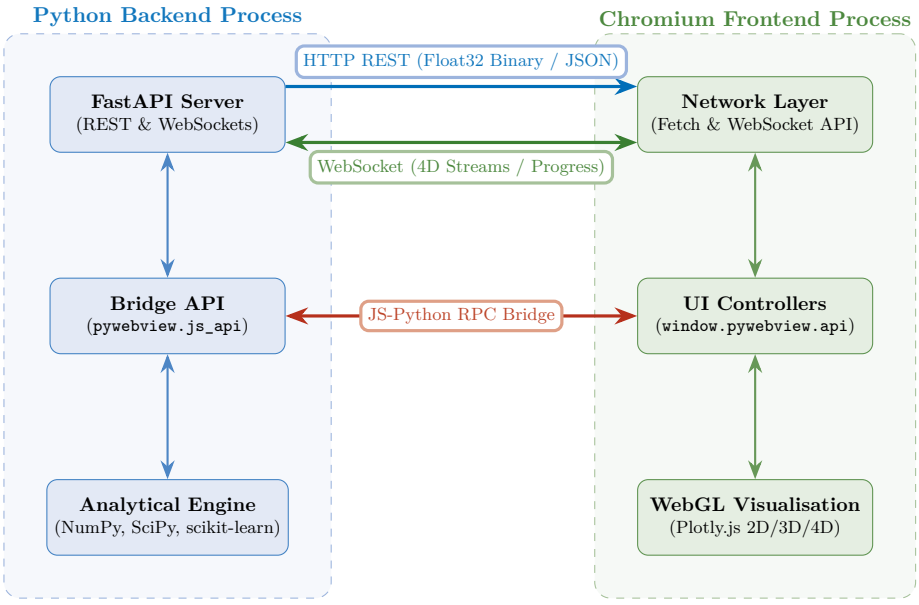


Figure 3.1. Dual-process hybrid architecture of Seismic Viewer with dedicated communication channels.

The architecture separates responsibilities across three dedicated communication

channels, balancing responsiveness and throughput:

1. **Native JS-Python RPC Bridge**: This channel connects the user interface with the Python runtime for desktop interactions. When the application window initializes, pywebview exposes the methods of the Python `Api` class to the JavaScript environment via `window.pywebview.api`. User interface actions, such as selecting a file or opening dialogs, invoke these methods as asynchronous Remote Procedure Calls (RPC). The framework serializes the call across the process boundary and returns the result as a resolved JavaScript Promise, executing desktop commands without network overhead.

2. **FastAPI HTTP REST Endpoints**: This channel handles high-throughput data transfer between the backend and the browser. Rather than embedding large datasets into HTML pages, the frontend requests earthquake catalogs asynchronously using REST endpoints. Numerical coordinates are streamed as contiguous Float32 binary buffers via `/api/data/{id}/bin`, while catalog metadata and pre-rendered visualization views are served with automatic GZip compression.

3. **FastAPI WebSockets**: This bi-directional channel manages real-time updates and interactive streaming during continuous operations. When running 4D temporal animations or clustering calculations, the backend streams progress notifications and frame updates directly to the client over an open socket connection. This allows the interface to update 3D scene elements and progress bars continuously without polling or page reloads.

Table 3.1 lists all REST endpoints and WebSocket message types supported by the system.

Table 3.1. REST API endpoints and WebSocket message protocol.

Protocol / Endpoint	Direction / Path	Description / Message Types
REST (`GET`)	`/api/data/{session_id}`	Retrieve session data as GZip-compressed JSON
REST (`GET`)	`/api/data/{session_id}/bin`	Retrieve numerical data as Float32 binary buffer
REST (`GET`)	`/viz/{viz_id}`	Serve pre-rendered HTML visualisation page
WebSocket	Client → Server	`ping`, `get_cluster_update`, `get_structural_planes`, `get_animated_clustering`
WebSocket	Server → Client	`pong`, `rpc_response`, `progress` (broadcast messages)

3.2 Backend Architecture

3.2.1 Application Entry Point & Bridge Engine

Once the application is running, the `Api` module acts as the central coordinator. Whenever the user interacts with the interface, such as selecting a file (`select_csv()`), loading a 3D view (`load_view()`), or exporting results (`export_clusters_csv()`), the frontend sends a request directly to this coordinator.

These requests include spatial analysis calculations on seismic events. To prevent blocking the graphical interface during execution, operations run on background workers using the `threading.Thread` module or process pools. The application uses `asyncio.run_coroutine_threadsafe` to send progress updates from worker threads to the WebSocket event loop without race conditions.

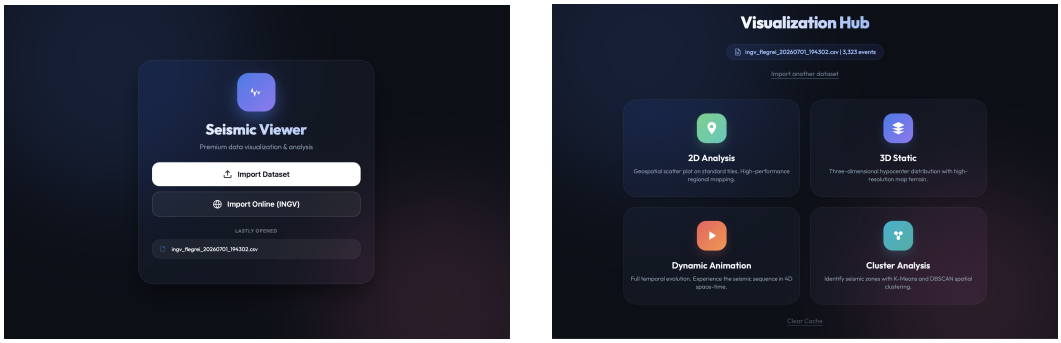
Since the calculations occur in the background, the application requires a mechanism to inform the user about the ongoing progress. The coordinator handles this requirement by continuously transmitting real-time updates back to the interface. This ensures the user is constantly provided with a responsive progress bar or status message, resulting in a fluid and stable user experience. These updates are broadcast through the WebSocket protocol.

3.2.2 Session Store & FastAPI Middleware

The system architecture requires a dedicated memory management layer to preserve state between user interactions and to serve data efficiently to the graphical frontend. This functionality is centralized within an internal configuration module (`core/config.py`), which orchestrates two separate in-memory caches: `_data_store` and `_html_store`.

The `_data_store` structure retains the raw numerical arrays and metadata associated with each active analytical session. To guarantee that the application maintains a stable memory profile even during extensive analytical workflows, this cache is strictly limited to 30 concurrent sessions. In parallel, the `_html_store` caches up to 15 pre-rendered graphical views, allowing the frontend to load complex tridimensional scenes instantaneously. Both structures implement a First-In, First-Out (FIFO) eviction policy. To ensure thread-safety during concurrent operations, updates to these stores are synchronized, maintaining application RAM usage below 500 MB during extended sessions.

Positioned between this caching layer and the client interface, the FastAPI server deploys specific middleware to optimize network throughput. An important optimization is the automatic application of GZip compression for any outgoing data payload larger than 500 bytes. Compressing the data prior to transmission accelerates the rendering process, particularly when transferring datasets containing tens of thousands of seismic events.



(a) File Picker view. **(b)** Visualisation Hub with mode selection.

Figure 3.2. The Seismic Viewer single-page application structure, showing initial data loading (a) and the multi-mode analytical selection hub (b).

3.3 Frontend Architecture

3.3.1 Single-Page Application & UI/UX Design

As illustrated in Figure 3.2, the primary workflow begins at the landing screen (a). Here, the system exposes multiple data ingestion pathways, allowing researchers to import local csv files, query the INGV remote web services directly, or restore a previously interrupted session through local historical records.

Once the imported dataset is validated, the software transitions to the visualization hub (b). This analytical workspace features menus to alternate between the available visualization and analysis algorithms. In addition, it incorporates interactive toggles to overlay supplementary spatial context onto the 3D map, allowing researchers to integrate seismic tomography models, GNSS deformation grids, and local monitoring stations visually.

The 3D rendering pipeline delegates plotting operations to the WebGL graphics API via Plotly.js. To preserve context during dynamic operations, the rendering layer maintains the spatial orientation and zoom level chosen by the user. This prevents the virtual camera from resetting its position when new data arrives, allowing the system to update the scene in real time without interrupting interaction (Section 7.2.1).

3.4 Data Flow Pipeline and Protocols

To manage data processing and rendering, Seismic Viewer divides operations between the backend services and the frontend client.

On the backend, the architecture handles data ingestion, validation, and mathematical calculations. As shown in Figure 3.3, the pipeline begins by importing CSV data or querying the INGV API. This data undergoes sanitisation and loading. Visual pre-computation prepares coordinate structures before the engine executes the analytical algorithm chosen by the user. Finally, results are assembled into an HTML template and stored in the internal `_data_store` and `_html_store` caches.

Once the backend processing is complete, the frontend retrieves the data through

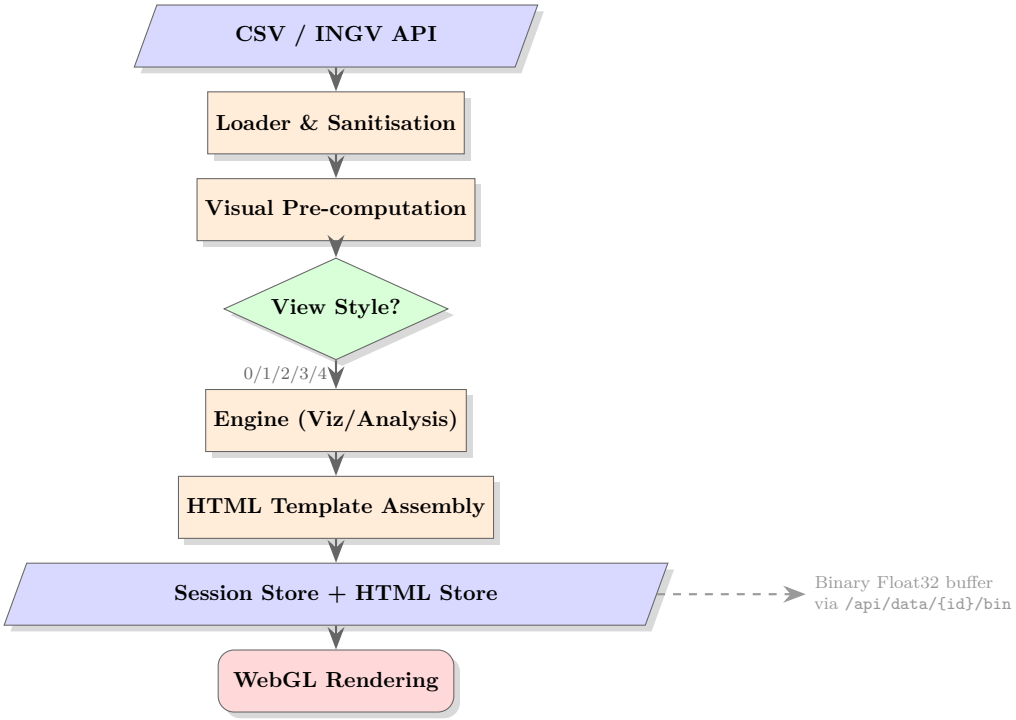


Figure 3.3. Linear data pipeline from input parsing to WebGL drawing.

the defined REST API and WebSocket endpoints (Table 3.1). The frontend dynamically renders the 3D scene using WebGL via the Plotly.js library. Continuous status updates, real-time recalculations, and filtering commands are exchanged over the WebSocket channel, ensuring the interface remains fully interactive without needing page reloads.

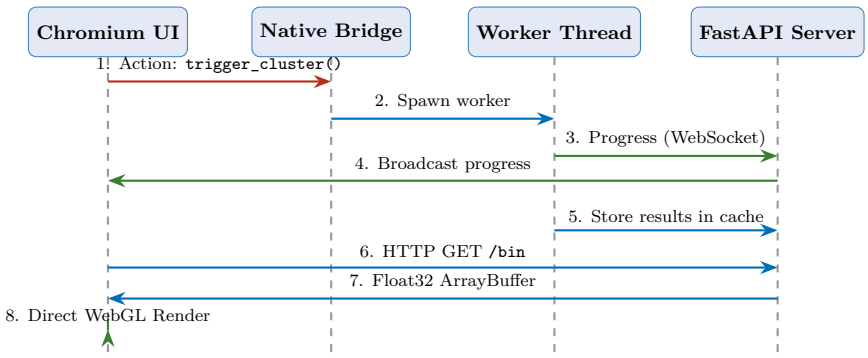


Figure 3.4. Sequence of asynchronous execution and non-blocking streaming pipeline.

3.4.1 Binary Data Transfer Protocol

To eliminate JSON parsing overhead during large catalog transfers, the backend packs numerical coordinates into a contiguous Float32 buffer (`/api/data/{id}/bin`) structured as: header `[event_count, cluster_count]` followed sequentially by

arrays for longitude, latitude, magnitude, depth, `dates_num`, and cluster labels (C floats).

Table 3.2. Binary buffer layout (Float32).

Offset	Content	Offset	Content
0 (2 floats)	Header: `[N, C]`	$2+3N$ (N floats)	Depth array
2 (N floats)	Longitude array	$2+4N$ (N floats)	Dates_num array
$2+N$ (N floats)	Latitude array	$2+5N$ (C floats)	Cluster labels (if present)
$2+2N$ (N floats)	Magnitude array		

On the frontend, decoding via JavaScript's `Float32Array` directly over an `ArrayBuffer` completes in under 5 ms for 50,000 events (reducing transfer volume by approximately 90% and bypassing over 800 ms of JSON parsing and garbage collection delays).

3.5 Threading and Concurrency Model

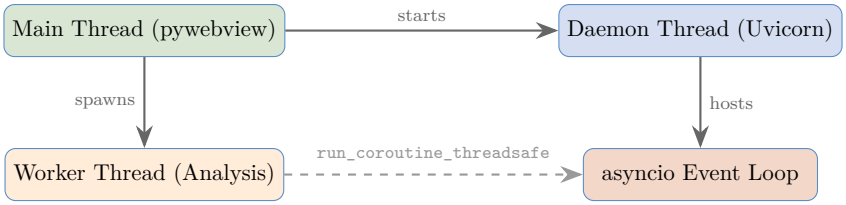


Figure 3.5. Concurrency hierarchy linking UI, server thread, and analysis worker threads.

The concurrency architecture separates responsibilities across the **Main Thread** (hosting pywebview and Chromium), the **Server Thread** (running the Uvicorn [1] asyncio event loop), and on-demand **Worker Threads** handling view generation and clustering updates. For intensive multi-frame calculations (animated clustering and RANSAC+SVD plane tracking), the engine escalates to a `ProcessPoolExecutor` with up to 8 worker processes utilising shared memory initialisers to avoid array serialisation. This hybrid concurrency model completely prevents contention during numerical loops, ensuring the GUI remains at 60 FPS even during multi-second plane fitting routines.

Chapter 4

Data Processing and Geospatial Engine

This chapter details the processing pipeline that transforms raw seismic observations (whether sourced from local catalogs or remote web services) into optimized, GPU-ready structures for interactive 3D and 4D visualization.

4.1 CSV Ingestion and Schema Detection

The `core/loader.py` module handles different formats through an ingestion pipeline that checks the header line for semicolon (`;`) or comma (`,`) delimiters, supporting European standards (INGV) as well as international formats (USGS, ISC). To avoid manual preprocessing, `core/config.py` defines an alias dictionary:

```
CSV_SCHEMA = {
    'time':  ['time', 'date', 'datetime', 'utc', 'time_utc', '
    origintime'],
    'lat':   ['lat', 'latitude'],
    'lon':   ['lon', 'longitude'],
    'depth': ['depth', 'depth_km'],
    'mag':   ['mag', 'magnitude', 'md', 'ml', 'mw']
}
```

Listing 4.1. Flexible column schema definition in `core/config.py`.

The ingestion pipeline identifies data columns by reading their headers. It ignores text case and removes any extra spaces. When a file contains unlabelled columns, the loader assumes a standard INGV column order based on their positions. Date processing is equally robust. The engine tests multiple known formats, covering both standard ISO timestamps and common European layouts.

4.2 Data Sanitisation and Validation

Following ingestion, records undergo a four-stage sanitisation and validation sequence:

1. **Spatial requirement check**: Rows missing latitude, longitude, or timestamps are discarded immediately.

2. **Missing metadata flagging**: Catalogues occasionally omit depth or magnitude. In seismic analysis, estimating missing depths (such as using the mean value) is avoided because it groups points into artificial horizontal layers, distorting clustering and plane fitting results. Rather than dropping these records (which removes spatial density) or imputing synthetic values, missing fields are flagged via boolean indicators (`depth_is_missing`, `mag_is_missing`) and assigned sentinel defaults (0.0 km depth and magnitude 0.1).

3. **Geographic and depth bounds check**: The system excludes any event with mathematically invalid geographic coordinates (i.e., latitude outside the $\pm 90^\circ$ range or longitude outside the $\pm 180^\circ$ range). Additionally, it discards events with a depth falling outside the realistic geological range of -10 km to 800 km, where negative values are specifically permitted to accommodate earthquakes occurring above sea level within mountains or volcanic edifices.

4. **Chronological sorting & future filtering**: Events with future timestamps are discarded, and the remaining catalog is sorted chronologically to optimize slicing.

4.3 Online Data Import: The INGV GOSSIP API

The `core/online_importer.py` module interfaces directly with the INGV GOSSIP web service [7], providing high-resolution volcanic monitoring catalogues for Campi Flegrei, Vesuvius, and Ischia:

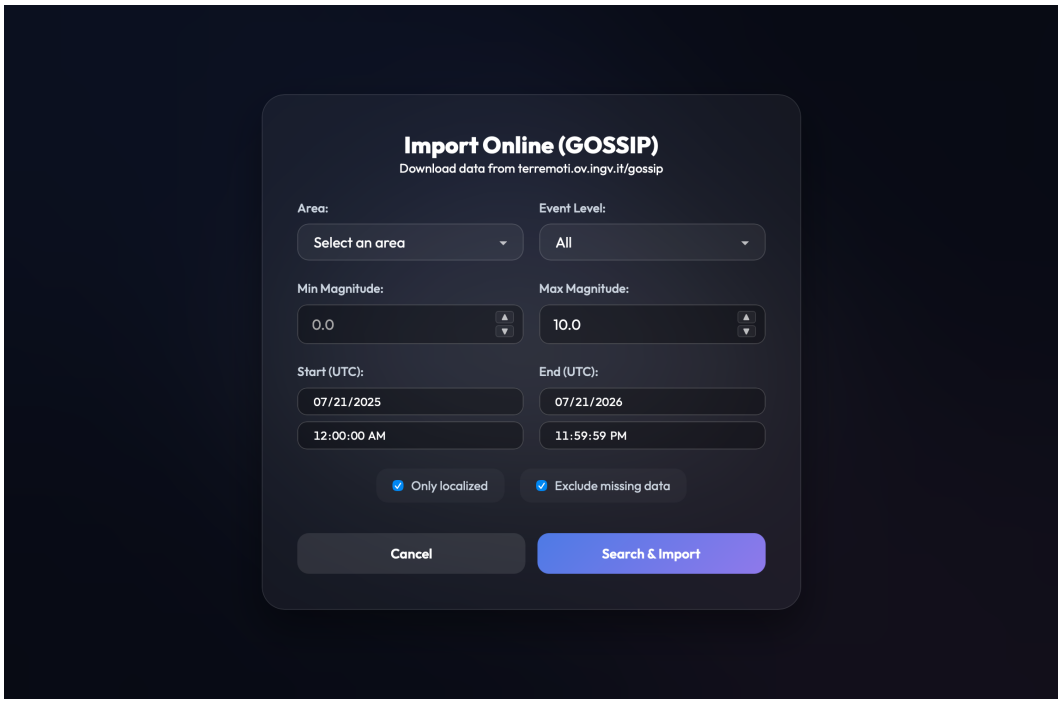


Figure 4.1. The INGV GOSSIP API import dialog for configuring network criteria, temporal bounds, and quality thresholds.

The importer queries the official INGV GOSSIP web portal for each selected year (using the endpoint `https://terremoti.ov.ingv.it/gossip/<network>/<year>/events.json`). It downloads these annual JSON packets one by one to avoid network failures. As shown in Figure 4.1, users can define the network target, the time span, the magnitude limits (M_{min} to M_{max}), and the desired INGV revision quality (`automatico`, `rivisto`, or `bollettino`). Users can also apply additional filters to require confirmed coordinates (`located only`) or to drop records with incomplete information (`exclude missing`). Finally, the system converts these filtered records into a standard temporary CSV format and forwards them directly to the main ingestion pipeline.

4.4 Geospatial Computations and Mesh Generation

The 3D application visualises seismic activity over a base map retrieved from OpenStreetMap [11]. To accurately draw the earthquakes over this map, the software must convert the image pixel positions into standard geographic coordinates. The system solves this problem using an **inverse Mercator projection** [19] (implemented in `core/geo_utils.py`). This mathematical operation translates pixel data into latitude and longitude, using NumPy [6] arrays to guarantee fast execution:

$$\lambda = \frac{x \cdot s}{t \cdot n} \cdot 360^\circ - 180^\circ, \quad \phi = \arctan\left(\sinh\left(\pi\left(1 - \frac{2 \cdot y \cdot s}{t \cdot n}\right)\right)\right) \cdot \frac{180^\circ}{\pi} \tag{4.1}$$

In these equations, λ and ϕ represent the resulting longitude and latitude in degrees. The values x and y correspond to the horizontal and vertical pixel coordinates on the screen. The variable s is the current scaling factor of the map, while t represents the fixed dimension of a single map tile (256 pixels). Lastly, n is the total number of tiles across the map at the current zoom level z, calculated as $n = 2^z$.

4.4.1 UTM Zone 33N to WGS84 Conversion

Some advanced datasets, such as volcanic tomography grids, are originally provided in UTM Zone 33N coordinates instead of standard latitude and longitude. To ensure these layers align correctly with the rest of the 3D map, the `core/layer_utils.py` module converts them using a Transverse Mercator inverse projection on the WGS84 ellipsoid:

$$\phi = \phi_1 - \frac{N_1 \tan \phi_1}{R_1}\left(\frac{D^2}{2} - \frac{(5 + 3T_1 + 10C_1 - 4C_1^2 - 9e'^2)D^4}{24}\right) \tag{4.2}$$

Here, ϕ is the final converted latitude. The formula relies on several variables derived from the WGS84 ellipsoid model. Specifically, ϕ_1 is the footprint latitude, N_1 is the prime vertical radius of curvature, and R_1 is the meridional radius of curvature at ϕ_1. The parameter D represents the scaled distance from the central meridian, calculated as $D = (E - 500{,}000)/(N_1 k_0)$, where E is the UTM Easting coordinate and k_0 is the central scale factor. Finally, T_1, C_1, and e'^2 are standard geometric constants related to the Earth's eccentricity.

4.4.2 Optimised Map Mesh and Caching Architecture

Terrain generation operates via a five-stage sequence: (1) bounding box squaring for uniform axis resolution, (2) deterministic cache lookup via MD5 hashing of rounded geographic bounds, (3) OpenStreetMap tile downloading via Smopy [15] on cache misses, (4) palette-indexed downsampling to 1024×1024 px (256 colours), and (5) grid transformation via Equation (4.1). Meshes are cached both in RAM (`@lru_cache(maxsize=4)`) and disk (`.pkl`), guaranteeing sub-millisecond retrieval on subsequent loads.

4.5 Pre-computation and Memory Layout Transformation

To ensure the 3D map remains fluid and responsive even with thousands of earthquakes, the software performs several calculations in advance (during the data loading phase) rather than while the user is interacting with the map. The `precompute_visual_data()` function handles two main tasks. First, it pre-renders tooltip strings in a single batch operation, so that the browser does not need to construct the text for pop-ups every time the mouse hovers over an event. Second, it converts calendar dates into normalised time metrics (days since the first earthquake), accelerating chronological color mapping.

The data pipeline converts raw CSV lines into contiguous NumPy numerical arrays (`numpy.ndarray`). This enables vectorised operations on coordinates, depths, and timestamps. To stream data to the client efficiently, arrays are converted to 32-bit floating-point values (`np.float32`) and transmitted as raw bytes via `tobytes()`, avoiding JSON encoding overhead.

Embedding large amounts of earthquake data directly into an HTML file increases file size and causes the browser to lag. To prevent this, Seismic Viewer uses an asynchronous loading strategy. When the user requests a 3D view, the backend generates an HTML page containing the 3D environment, base terrain maps, and UI menus. Once the base map is displayed, the browser requests the earthquake points in the background via `/api/data/{id}/bin`. These points are delivered as a Float32 binary buffer and populated onto the map without blocking user interaction.

Chapter 5

Visualisation Modes

Seismic Viewer provides four primary visualisation modes, each addressing a distinct analytical need. This chapter documents their implementation, rendering strategies, and integration with auxiliary geophysical data layers.

5.1 2D Geospatial Map View

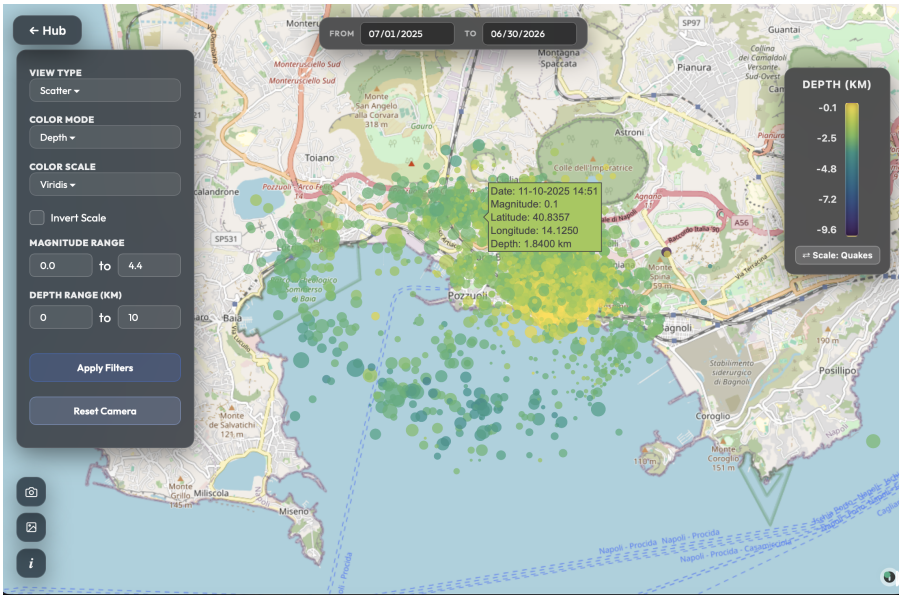
The 2D view provides a plan-view representation of earthquake epicentres overlaid on OpenStreetMap [11] tiles, implemented through Plotly's `scatter_mapbox` trace type.

5.1.1 Map Layers and Real-Time Filtering

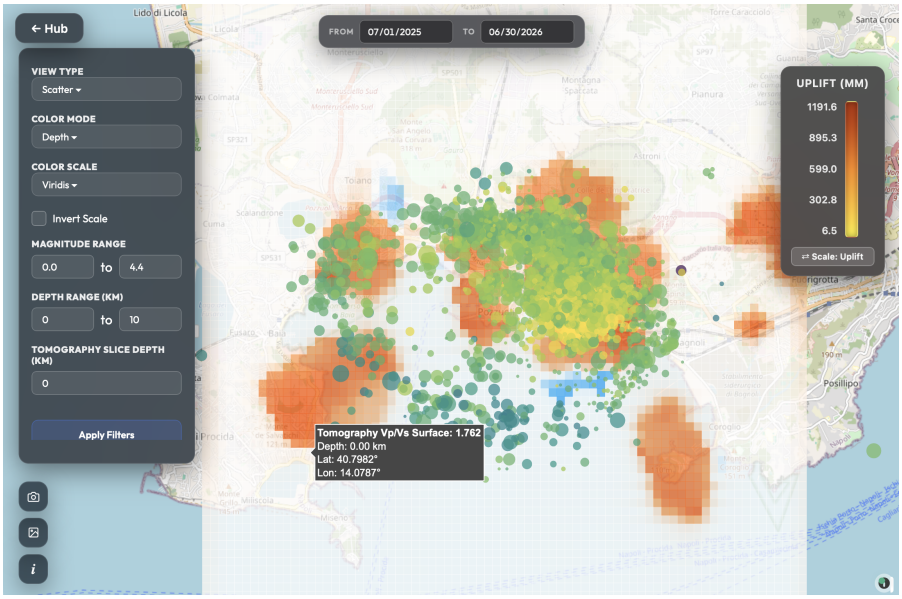
The 2D map visualizes the seismic data using two distinct graphical layers:

- **Scatter Markers**: Each earthquake is represented by a circular marker. The visual properties of these circles can be mapped to different physical parameters depending on the analytical needs. By default, the size of the circle indicates the magnitude of the event (with a minimum size threshold to ensure extremely weak micro-earthquakes remain visible), and the color represents the hypocentral depth. However, these mappings are highly flexible, allowing users to visualize other variables, such as chronological progression. Users can interact with these markers by hovering over them to reveal detailed statistics.
- **Density Heatmap**: A spatial density layer that visually aggregates tightly clustered earthquakes. By rendering a continuous color gradient over regions of high seismicity, this optional overlay allows researchers to intuitively identify active swarm locations and broader structural trends that might be difficult to perceive when analyzing individual scatter points alone.

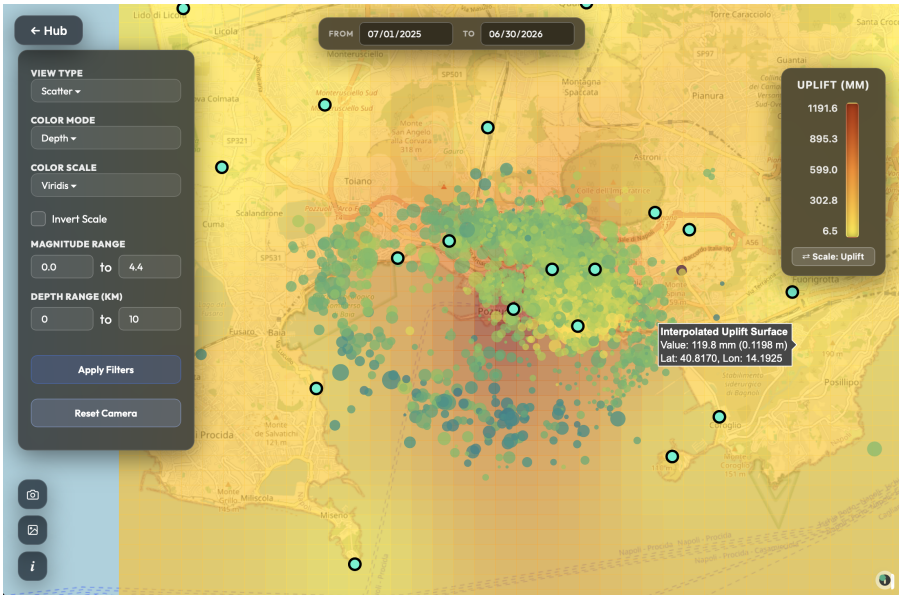
To explore the dataset, users can apply filters based on magnitude, depth, and date ranges using dedicated input fields located on the left-hand panel. All filtering operations are processed directly within the user's web browser. This approach avoids sending new requests to the main server, guaranteeing that the map updates instantly whenever a parameter is changed.



(a) The 2D base map showing earthquake locations. On the left panel, input fields allow the user to filter the data by magnitude, depth, and date.



(b) An overlay showing a tomography slice of the crust. The highlighted areas indicate high V_p/V_s velocity anomalies.



(c) An overlay displaying vertical ground movement (uplift) using a grid map, along with the locations of permanent monitoring stations across Campi Flegrei.

Figure 5.1. The 2D Geospatial Map View in the Seismic Viewer. This workspace allows users to filter earthquake data (a) and combine it with advanced geophysical overlays, such as tomographic velocity maps (b) and ground uplift measurements (c).

As shown in Figure 5.1, the 2D workspace offers three visualisation options. The base view (Figure 5.1a) displays earthquake locations on an OpenStreetMap layer, while the side panel allows users to filter events by magnitude, depth, and date. In Figure 5.1b, a tomographic slice shows variations in the V_p/V_s velocity ratio, indicating structural limits and underground fluid bodies. Finally, in Figure 5.1c, the map combines monitoring stations with vertical ground movement, helping researchers see if earthquakes match areas of surface uplift.

5.2 3D Static View

The 3D static view renders hypocentres as a subsurface point cloud within a three-dimensional geographic volume beneath an OpenStreetMap terrain surface. The reference terrain is rendered as a flat, unshaded map layer for clarity, while earthquakes are represented beneath the surface as a 3D point cloud, colour-coded by variables like depth or time.

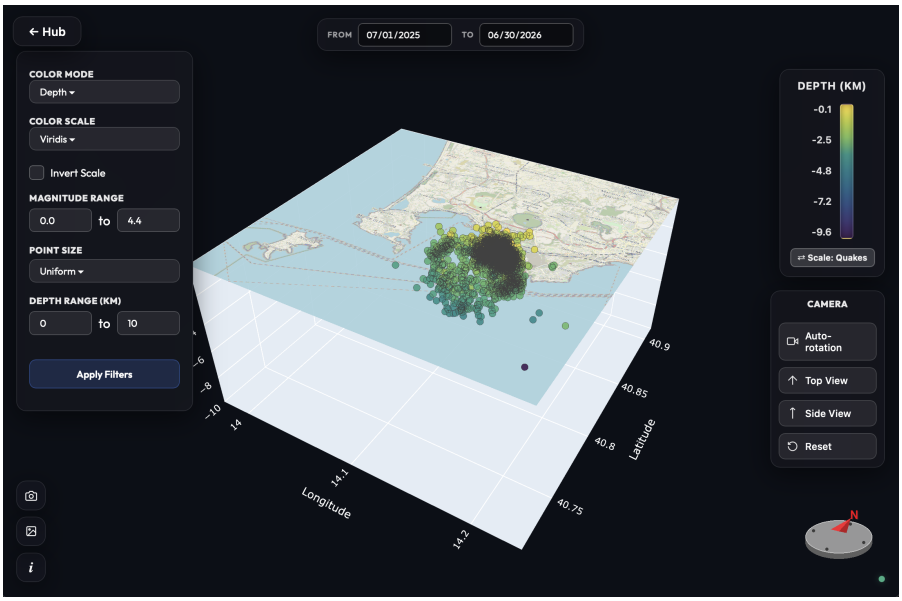
To prevent geographic distortion caused by Earth's sphericity, the system scales horizontal proportions based on latitude:

$$\text{ratio}_x = \frac{\Delta\lambda \cdot K \cdot \cos(\bar{\phi})}{\max(d_{\text{lon}}, d_{\text{lat}})}, \quad \text{ratio}_y = \frac{\Delta\phi \cdot K}{\max(d_{\text{lon}}, d_{\text{lat}})} \tag{5.1}$$

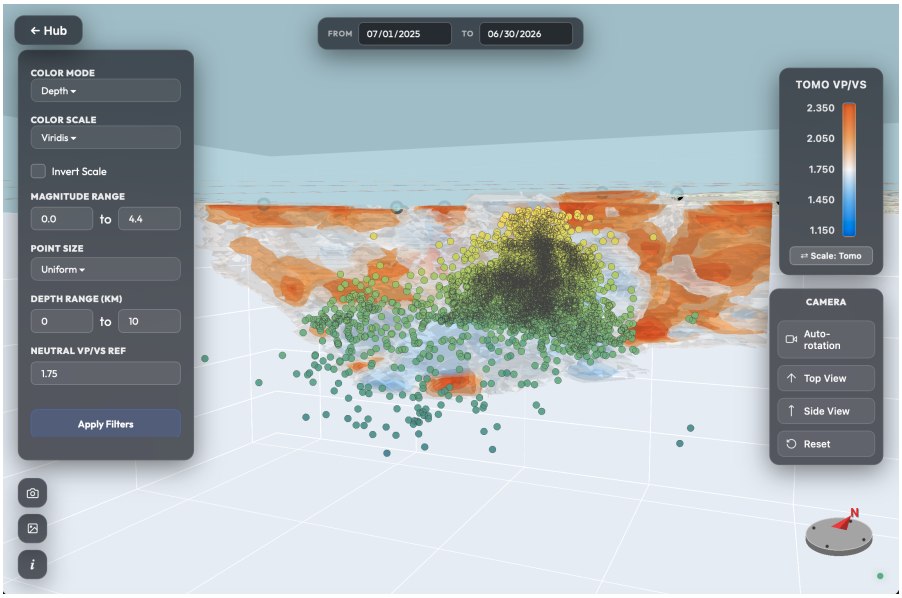
where $\Delta\lambda$ and $\Delta\phi$ are the longitude and latitude spans, $K = 111.32$ km/° is the equatorial conversion factor, and $\bar{\phi}$ is the mean latitude. This adjustment preserves accurate spatial proportions.

Rendering 50,000 3D points simultaneously relies on GPU acceleration. Plotly.js uses WebGL to batch point drawing via the `scatter3d` trace type. To maintain a 60 FPS frame rate, the application applies three measures: (1) map background images are downsampled to save memory, (2) automatic bounding box recalculations are disabled during trace updates, and (3) markers are drawn as 2D point sprites facing the camera rather than 3D meshes.

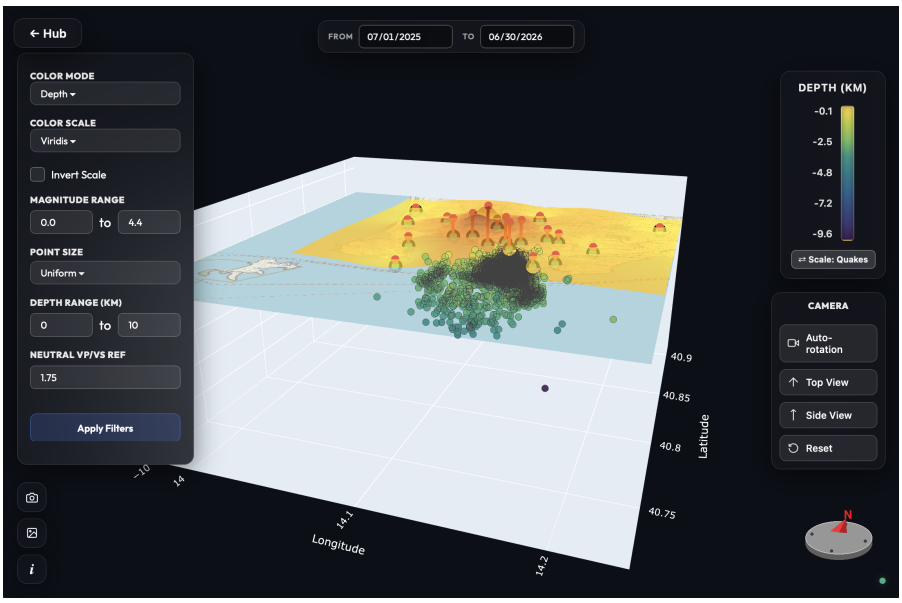
The 3D workspace (Figure 5.2) enables visualization of earthquake cluster geometries. Integrating a 3D tomography model (Figure 5.2b) correlates seismicity with geological structures, while vertical uplift columns (Figure 5.2c) allow direct comparison between surface deformation and underlying seismic activity.



(a) Point cloud below surface map.



(b) Added tomography layer.



(c) Ground uplift columns.

Figure 5.2. The 3D Static View workspace. The user can explore the underground earthquake distribution (a) and combine it with 3D tomography models (b) and surface deformation data (c).

As shown in Figure 5.2, the 3D static workspace presents these data layers in three dimensions. The base view (Figure 5.2a) shows hypocentres below the map surface, using colours to represent depth intervals. In Figure 5.2b, the scene adds velocity slices from the tomography model. Finally, in Figure 5.2c, vertical columns indicate surface movement at each station, allowing direct comparisons between

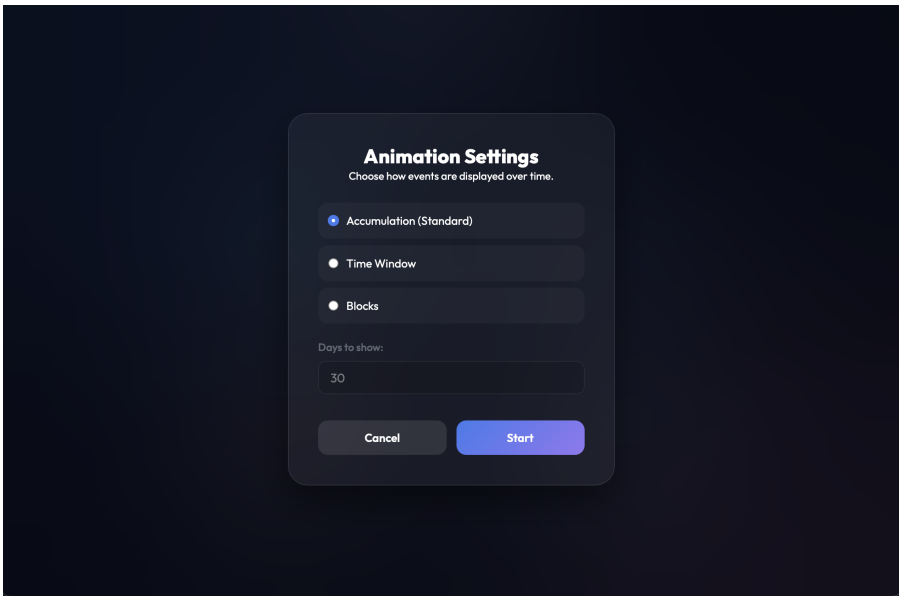
depth and surface observations.

5.3 3D Animated View (4D Visualisation)

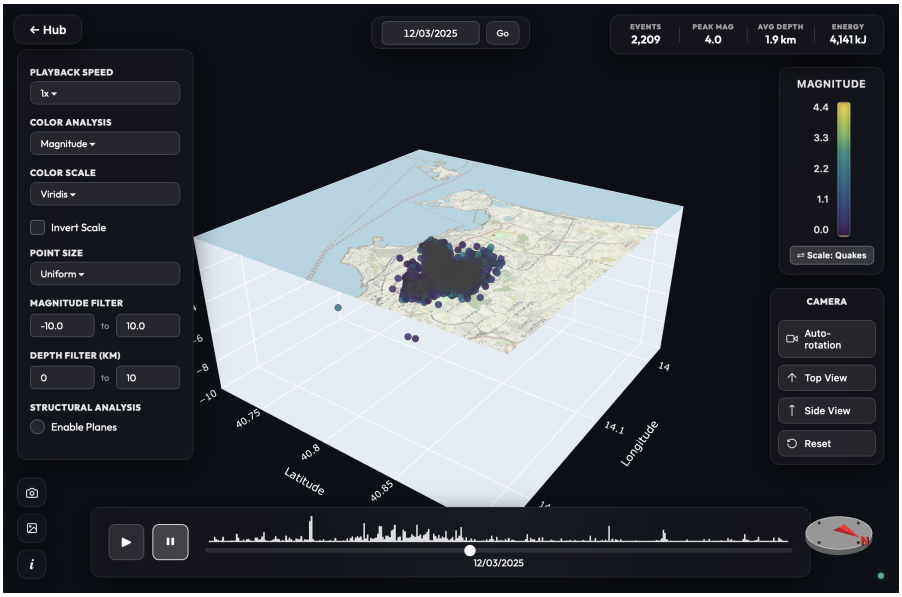
The animated view adds time as a fourth dimension to observe the spatio-temporal evolution of seismic sequences.

The 4D visualisation engine offers three temporal modes to analyse seismic evolution: **Accumulation** (showing all events up to the current frame), **Sliding window** (displaying events within a moving time frame), and **Block mode** (rendering discrete, non-overlapping time intervals). To ensure smooth playback and avoid memory overload, the system precomputes temporal indices and updates the 3D scene dynamically without reloading the entire figure.

During 4D animations, the camera angle and zoom level must remain stable across updates. By default, Plotly reinitializes the view on coordinate changes. Setting `uirevision='constant'` in the layout preserves camera position and orientation across frames, allowing continuous interaction during playback.



(a) Temporal window configuration dialog.



(b) Active 4D playback view showing the time-sliced hypocentral point cloud.



(c) Dynamic cylinder grid partitioning with automated structural plane fitting during 4D evolution.

Figure 5.3. The 4D Animated Spatio-Temporal View workflow. Users configure parameters via the dialog (a), explore time-evolving seismicity (b), and observe automated fault plane tracking (c).

Prior to starting the animation, the time parameters (such as step size and frame rate) are configured via a dedicated dialog window (Figure 5.3a). The resulting 4D view (Figure 5.3b) allows interactive exploration of the chronological sequence. In addition, the volume can be partitioned into a spatial grid of vertical cylinders (Figure 5.3c). This subdivision was implemented to better understand the spatial distribution of the data, as partitioning the volume effectively isolates and subdivides the complex fault geometry into local, simpler planar segments. As earthquakes occur inside these cylinders, the system dynamically calculates and renders these local structural fault planes in real-time, allowing users to visualize how seismic events evolve over time across the identified planar structures.

5.4 3D Cluster Analysis View

The cluster analysis mode integrates spatial machine learning with structural geological modeling.

5.4.1 Cluster Visualisation and Interactive Tuning

To provide a comprehensive geometrical understanding of seismogenic structures, Seismic Viewer renders every identified cluster through four complementary representations within the 3D WebGL scene. These representations were chosen to address different analytical needs: boundary delineation, volumetric extent, structural orientation, and individual event membership. Together, they allow researchers to examine a cluster from multiple geometric perspectives simultaneously.

1. **Wireframe Convex Hulls (`Scatter3d`)**: Outer boundary envelopes highlighting the spatial footprint and volumetric extent of each distinct cluster.
2. **Triangulated Solid Meshes (`Mesh3d`, `alphahull=0`)**: Smooth, semi-transparent volumetric convex hulls enclosing the clusters.
3. **Structural Fault Planes (`Mesh3d`)**: Planar surfaces calculated via RANSAC robust estimation coupled with Singular Value Decomposition (SVD) refinement. Each plane displays orientation vectors alongside precise geological strike and dip angles upon mouse hover.
4. **Clustered Event Spheres**: Discrete event spheres colored by cluster label using categorical palettes, with unclustered background noise explicitly assigned neutral grey.

When users interactively adjust spatial clustering hyperparameters, such as neighborhood radius ε, minimum samples threshold `min_samples`, or cluster count K, via the side control panel, the frontend dispatches a `get_cluster_update` WebSocket message. The Python backend asynchronously recomputes cluster labels and structural planes without blocking the event loop, returning the newly generated Plotly trace definitions. The frontend then replaces the existing traces in the WebGL scene, thereby updating the visualization while preserving the user's current camera perspective.

5.4.2 Sub-Clustering

To analyze hierarchical structures, researchers can perform secondary sub-clustering on specific data subsets. By utilizing the **locked clusters** feature, users can preserve previously identified and validated groups. The algorithm will then selectively re-analyze and sub-divide all the remaining unlocked clusters simultaneously. The refined results can subsequently be applied back to merge cleanly into the parent view.

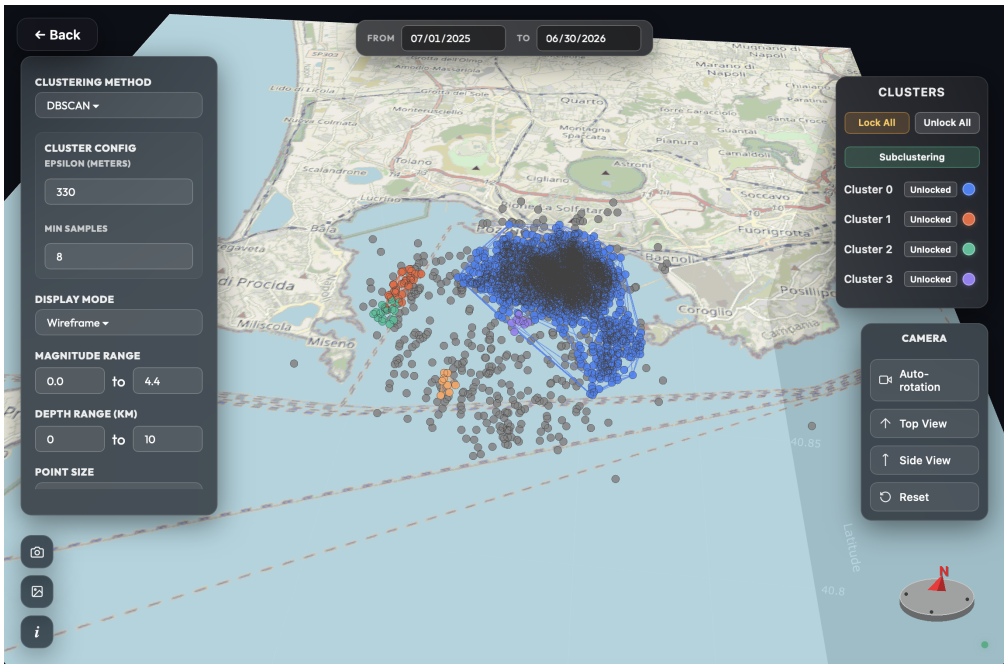
Figure 5.4 provides a comprehensive overview of the clustering tools. Figure 5.4a shows the standard 3D clustering interface. Figure 5.4b displays how clustering evolves over time. To ensure visual consistency, a tracking algorithm guarantees that each cluster maintains the same color across all animation frames. Finally, Figure 5.4c and Figure 5.4d combine the animated clusters with dynamically calculated fault planes and spatial cylinders. This combination allows researchers to easily observe how geological structures evolve and rotate over time.

5.5 Additional Data Layers & View Caching

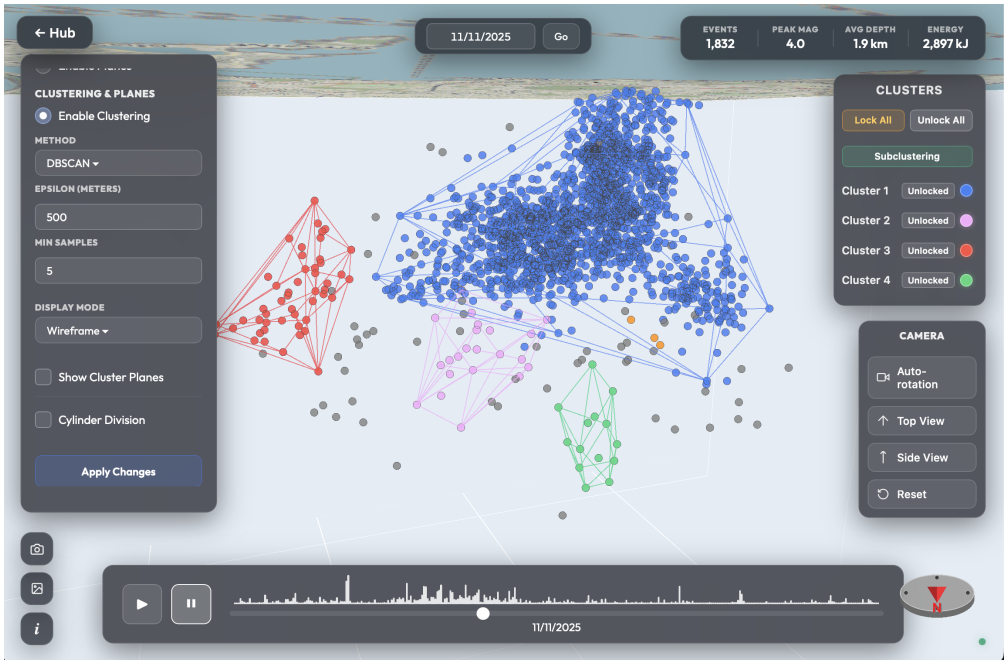
5.5.1 Auxiliary Layers: Tomography, Uplift, and Stations

Seismic Viewer integrates three auxiliary data types processed via `core/layer_utils.py`:

- **Seismic Tomography (V_p/V_s Ratio)**: The raw tomographic data, originally provided in UTM Zone 33N coordinates, is converted to geographic coordinates and mapped onto a dense $60 \times 50 \times 40$ 3D grid. Because measurements only



(a) Static 3D workspace with hulls and planes.

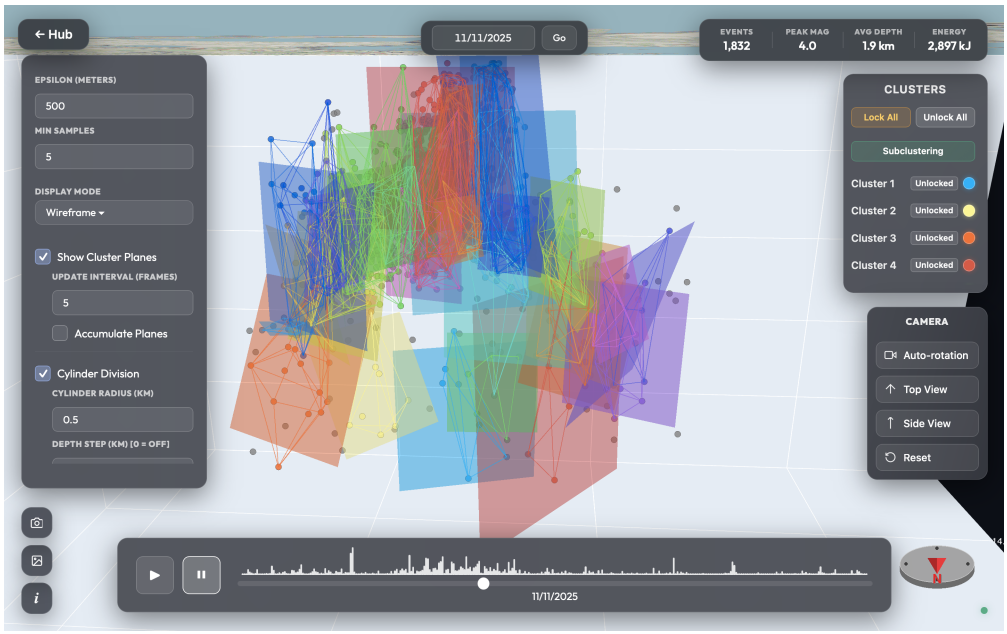


(b) Animated clustering with Global Label Matching.

Figure 5.4. Multi-scale static (a) and animated (b to d) 3D/4D Cluster Analysis modes (Part 1).



(c) Animated clustering with RANSAC+SVD planes.



(d) Unified 4D cylinder partitioning and plane fitting.

Figure 5.4. Multi-scale static (a) and animated (b to d) 3D/4D Cluster Analysis modes (Part 2).

exist at specific surveyed points, the software must estimate the values for the surrounding empty space. It achieves this using *Inverse Distance Weighting*, a spatial interpolation technique. The value of any unknown point is calculated as a weighted average of its $k = 16$ nearest known neighbors. The closer a neighbor is, the stronger its influence on the final calculation, scaling inversely with the square of the distance ($p = 2$):

$$V(\mathbf{x}) = \frac{\sum_{i=1}^{k} w_i \cdot V_i}{\sum_{i=1}^{k} w_i}, \quad w_i = \frac{1}{d(\mathbf{x}, \mathbf{x}_i)^2} \tag{5.2}$$

However, Inverse Distance Weighting can produce unrealistic mathematical artifacts in empty regions located very far from actual measurement stations. To solve this, the system applies a *distance-decay* function, which acts as a spatial fade-out effect. As the distance from the nearest station increases, this function smoothly blends the estimated value back into the standard natural background value of the Earth's crust (1.75):

$$V_{\text{final}}(\mathbf{x}) = \alpha(\mathbf{x}) \cdot V_{\text{IDW}}(\mathbf{x}) + (1 - \alpha(\mathbf{x})) \cdot 1.75 \tag{5.3}$$

where $\alpha(\mathbf{x}) = \exp(-(d_{\min}/\sigma)^2)$ with $\sigma = 0.15$ acts as the blending weight, decreasing smoothly from 1 (near known data) to 0 (far away).

- **GNSS Ground Deformation (Uplift)**: To visualize ground movement, the system reads historical displacement records and matches them with the known locations of the monitoring stations. This provides the exact vertical movement at any given time. Because data is only available at these specific points, the software estimates the ground movement in the empty areas between them using the same distance-based averaging (IDW) described above. The final result is displayed on the map either as a colored 2D grid or as 3D columns rising from the surface.
- **Monitoring Stations**: Permanent station positions are plotted as labelled markers, aiding spatial evaluation of catalogue completeness thresholds.

These three auxiliary layers serve complementary roles in the interpretation workflow. By overlaying tomographic anomalies and ground deformation patterns onto the same 3D scene as the earthquake hypocentres, researchers can visually assess whether seismic swarms correlate with known velocity anomalies or with areas of maximum surface uplift, providing a more integrated understanding of the underlying volcanic processes.

5.5.2 View Caching and Navigation

To ensure faster navigation between complex workspaces, the backend bridge maintains an in-memory view cache (`_html_store`, max 15 entries) keyed by a composite tuple of file path, view style, animation/windowing parameters, clustering configuration, and locked cluster states. A navigation history stack (`_history_stack`) allows instantaneous back and forward navigation in the clustering view while preserving exact parameter context.

Chapter 6

Clustering and Structural Analysis Algorithms

This chapter formalises the algorithms used in Seismic Viewer's analytical engine, presenting their mathematical formulations and optimization strategies.

6.1 Coordinate Projection and Spatial Clustering Methods

To compute Euclidean distances for spatial clustering, spherical coordinates (λ, ϕ) are converted into a local 3D Cartesian coordinate system in kilometres $(x, y, z)_{\text{km}}$ centred at the dataset mean (λ_0, ϕ_0). This equirectangular approximation [19] is defined as:

$$x = (\lambda - \lambda_0) \cdot K \cos(\phi_0), \quad y = (\phi - \phi_0) \cdot K, \quad z = z_{\text{depth}} \tag{6.1}$$

where $K = 111.32$ km/° represents the average length of one degree of latitude, and z_{depth} represents the depth of each individual earthquake (measured as positive downwards).

6.1.1 DBSCAN, MiniBatchKMeans, and Cylinder Partitioning

Seismic Viewer implements three complementary clustering paradigms, each provided by the scikit-learn library [12] or developed as custom extensions:

1. **DBSCAN**: The density-based spatial clustering algorithm [3] (`DBSCAN`) isolates high-density seismogenic structures according to neighbourhood radius $\varepsilon_{\text{km}} = \varepsilon_{\text{m}}/1000$ and `MinSamples`. Points lacking sufficient density reachability are marked as noise (label -1).

2. **MiniBatchKMeans**: For centroid partitioning, the MiniBatchKMeans variant [9] optimises K clusters across random data subsets ($b = 1024$), reducing iteration complexity from $O(NK)$ to $O(bK)$. To prevent outlier inclusion, a post-clustering distance filter reclassifies points exceeding $d_{\max}$ from their centroid as noise:

```
if method == 'kmeans' and max_dist > 0:
    max_d = max_dist / 1000.0
    for i in range(n_clusters):
        idx = (labels == i)
        dists = np.sqrt(np.sum((pts_km[idx] - centroids[i])
    **2, axis=1))
        display_labels[np.where(idx)[0][dists > max_d]] = -1

```

Listing 6.1. Post-clustering distance filter for KMeans.

3. **Cylinder Partitioning**: To ensure stable cluster assignments during 4D temporal animations without color swapping, the study volume is partitioned into a regular 2D grid of fixed vertical cylinders (60×50 cells). Events are assigned to the nearest cylinder centroid via KDTree query. Cylinders accumulating more than 5 events constitute stable, depth-resolved clusters across frames.

Table 6.1. Comparison of spatial clustering approaches in Seismic Viewer.

Property	**DBSCAN**	**MiniBatchKMeans**	**Cylinder Partition**
Cluster count / shape	Automatic / Arbitrary	User-defined (K) / Spherical	Grid-dependent / Cylindrical
Noise handling	Native (label -1)	Distance threshold ($d_{\max}$)	Density threshold (≥ 5 events)
Primary use case	Unknown structures, swarms	Fixed centroid tracking	4D animation, fault plane tracking

6.2 Automatic Parameter Tuning

Parameter auto-tuning is executed during *Subcluster Analysis* to estimate parameters for unlocked clusters based on their spatial distribution. The method is based on the Kneedle algorithm [16], which detects the point of maximum curvature in a sorted distance or inertia curve.

DBSCAN Auto-ε. Density-based clustering depends on two parameters: the neighbourhood radius ε and the minimum number of points k = MinSamples. Because these parameters are interdependent, the analytical engine determines both automatically. First, the minimum sample threshold is computed from the dataset size N as $k = \max(3, \lfloor \ln N \rfloor)$, allowing the density requirement to adapt to the data volume. Next, using a k-nearest neighbours search, the algorithm calculates the Euclidean distance $d_i^{(k)}$ from each hypocentre to its k-th closest neighbour. Sorting these distances in ascending order yields the k-distance curve, denoted as points $\mathbf{p}_i = (i, d_i^{(k)})$ for $i = 0, \ldots, N-1$.

In spatial clustering theory, points belonging to seismic clusters exhibit lower k-distances, whereas boundary events and noise cause the curve to rise. The

neighbourhood radius ε^* corresponds to the knee point of this curve, which marks the transition between cluster density and noise. To identify this point, the system constructs a secant line between the first point $\mathbf{p}_0 = (0, d_0^{(k)})$ and the last point $\mathbf{p}_{N-1} = (N-1, d_{N-1}^{(k)})$, defining the direction vector $\mathbf{v} = \mathbf{p}_{N-1} - \mathbf{p}_0$. The algorithm calculates the orthogonal distance from each curve point $\mathbf{p}_i$ to the secant line and selects the index i^* that maximizes this distance:

$$i^* = \arg\max_{0 \le i < N} \left\| (\mathbf{p}_i - \mathbf{p}_0) - \left(\frac{(\mathbf{p}_i - \mathbf{p}_0) \cdot \mathbf{v}}{\|\mathbf{v}\|^2} \right) \mathbf{v} \right\| \tag{6.2}$$

The search radius is assigned as $\varepsilon^* = d_{i^*}^{(k)}$, providing a parameter that separates earthquake clusters from background noise.

MiniBatchKMeans Auto-K. For centroid-based partitioning, finding the number of clusters K requires balancing grouping compactness against model complexity. The analytical engine evaluates candidate values from $K = 2$ to $K = \min(10, N)$, computing the clustering inertia $J(K) = \sum_{j=1}^{K} \sum_{\mathbf{x} \in C_j} \|\mathbf{x} - \boldsymbol{\mu}_j\|^2$ for each configuration. Plotting $J(K)$ against K produces a decreasing curve. The cluster count K^* is identified by applying the orthogonal projection method described in Equation (6.2) to the inertia curve, locating the elbow point where adding clusters yields diminishing improvements in compactness.

6.3 Structural Plane Fitting and Geometric Synthesis

To extract planar fault structures from hypocentral distributions, Seismic Viewer uses a two-stage pipeline combining Random Sample Consensus (RANSAC) [4] and Singular Value Decomposition (SVD) [17]. RANSAC isolates points belonging to a plane while rejecting outliers and background seismicity. Once this subset of inliers is isolated, SVD computes the least-squares plane across the identified inliers and evaluates the planarity of the structure.

6.3.1 RANSAC Robust Plane Estimation

Seismic hypocentres rarely form geometric planes and contain background events or intersecting branches. Fitting a plane directly using least squares is skewed by outliers. RANSAC addresses this through an iterative consensus search. In each iteration, the algorithm samples a triplet of points $\{P_1, P_2, P_3\}$ and calculates a candidate normal vector:

$$\vec{n}_{\text{cand}} = \frac{(P_2 - P_1) \times (P_3 - P_1)}{\|(P_2 - P_1) \times (P_3 - P_1)\|} \tag{6.3}$$

The algorithm counts how many points in the cluster lie within a distance threshold of this candidate plane. Points meeting this condition are classified as inliers. This process is repeated across iterations, and the candidate model with the largest inlier count and lowest residual error is selected as the consensus set.

6.3.2 SVD Refinement and Strike/Dip Mechanics

Although RANSAC filters out outliers and isolates the inlier set, the candidate plane itself is defined by three sample points. To obtain the plane that accounts for all identified inliers, the engine applies Singular Value Decomposition (SVD).

The software first computes the centroid $(\bar{x}, \bar{y}, \bar{z})$ of all n inliers and centres the coordinates at the origin, constructing the data matrix $A \in \mathbb{R}^{n\times 3}$:

$$A = \begin{bmatrix} x_1 - \bar{x} & y_1 - \bar{y} & z_1 - \bar{z} \\ \vdots & \vdots & \vdots \\ x_n - \bar{x} & y_n - \bar{y} & z_n - \bar{z} \end{bmatrix} = U\Sigma V^T \tag{6.4}$$

The decomposition factors A into orthogonal singular vectors and singular values $\Sigma = \mathrm{diag}(\sigma_1, \sigma_2, \sigma_3)$, where $\sigma_1 \geq \sigma_2 \geq \sigma_3 \geq 0$. The right singular vector corresponding to the smallest singular value σ_3 represents the direction of minimum variance, which defines the normal vector $\vec{n}$ of the fitted plane.

In addition to defining the normal vector, the singular values quantify the geometric shape of the hypocentral cloud. The engine calculates a Planarity metric P:

$$P = \frac{\sigma_2 - \sigma_3}{\sigma_1} \tag{6.5}$$

A high planarity score indicates a planar fault segment, whereas a low score indicates a dispersed distribution of events.

To translate the normal vector $\vec{n} = [n_x, n_y, n_z]$ into standard geological metrics, the vector is oriented downwards ($n_z \geq 0$). Trigonometric formulas then extract the orientation of the fault:

$$\text{Dip}: \quad \delta = \arccos(n_z) \cdot \frac{180^\circ}{\pi} \tag{6.6}$$

$$\text{Dip Direction}: \quad \theta_{\mathrm{DD}} = \mathrm{atan2}(n_x, n_y) \bmod 360^\circ \tag{6.7}$$

$$\text{Strike}: \quad \phi_s = (\theta_{\mathrm{DD}} - 90^\circ) \bmod 360^\circ \tag{6.8}$$

The Dip measures the tilt angle relative to the surface, the Dip Direction indicates the geographic direction of the downward slope, and the Strike defines the orientation of the horizontal fault line according to the right-hand rule. For 3D visualization, planes are rendered as triangulated grids constructed along orthogonal basis vectors $\vec{u}, \vec{v}$:

$$\vec{u} = \frac{\vec{n} \times \hat{z}}{\|\vec{n} \times \hat{z}\|}, \quad \vec{v} = \frac{\vec{n} \times \vec{u}}{\|\vec{n} \times \vec{u}\|} \tag{6.9}$$

6.4 Temporal Stability, Parallel Execution, and Complexity

Computing structural models across temporal sequences requires visual consistency and computational efficiency. This section describes the temporal tracking mechanism, the parallel execution model, and the computational complexity of the analytical engine.

6.4.1 Temporal Stability via Global Label Matching

When running unsupervised clustering algorithms frame by frame during 4D animations, each time step evaluates an independent subset of seismic events. Because unsupervised algorithms assign cluster labels without historical context, the label of a fault structure can change between consecutive frames. In an animation, this causes colour changes (known as colour flickering), which prevents tracking how faults evolve over time.

To maintain colour continuity across frames, the engine uses a label matching algorithm. The system maintains an array recording the most recent label assigned to each earthquake. For every cluster detected in a new frame, the algorithm inspects the previous labels of its events and applies majority voting:

$$L_{\text{assigned}} = \arg\max_{k} \sum_{j \in S_c} \mathbb{I}(\text{global_map}[j] = k) \tag{6.10}$$

where S_c is a sample of indices from the cluster (bounded to 200 events), and $\mathbb{I}$ is the indicator function. If a cluster contains only new events without previous history, the algorithm assigns an incremental label. The assigned label is applied to colour the earthquakes in the active frame and saved in the global array so that subsequent frames reuse the same cluster identifier.

6.4.2 Parallel Execution Architecture

Calculating RANSAC inliers and SVD planes across animation frames involves separate mathematical steps for each time window. To prevent these calculations from slowing down the user interface, the backend distributes frame tasks across CPU cores using a process pool. In multi-process programs, copying coordinate arrays between processes creates overhead. The engine avoids this by loading the coordinate array into shared memory when the worker pool starts, allowing processes to read coordinates directly without making copies.

6.4.3 Asymptotic Complexity Analysis

Table 6.2 summarizes the time and space complexity of each algorithm in the pipeline. Spatial neighbor searches in DBSCAN use a KD-Tree index to reduce query complexity to $O(N \log N)$, while MiniBatchKMeans scales with mini-batch size rather than full dataset dimensions. Plane fitting scales with the number of inliers per cluster, and label matching remains bounded by sampling.

Table 6.2. Asymptotic time and space complexity of the analytical methods.

Algorithm / Method	Time Complexity	Space Complexity	Parameters & Notation
DBSCAN (KD-Tree)	$O(N \log N)$	$O(N)$	N events, ε neighbourhood radius
MiniBatchKMeans	$O(b \cdot K \cdot I)$	$O(b \cdot K)$	$b = 1024$ batch, K centroids, I iterations
Cylinder Partitioning	$O(N \log M)$	$O(M + N)$	M grid cylinders (60×50)
Kneedle Auto-Tuning	$O(N \log k + K_{\max} \cdot b)$	$O(N)$	k-NN ($k = \max(3, \lfloor \ln N \rfloor)$), $K \in [2, 10]$
RANSAC + SVD Fitting	$O(I_{\text{ransac}} \cdot N_{\text{cluster}} + d^3)$	$O(N_{\text{cluster}})$	$I_{\text{ransac}} = 100$, dimension $d = 3$
Global Label Matching	$O(K \cdot \min(N_c, 200))$	$O(N)$	Majority voting with bounded sampling

Chapter 7

Development Process and Challenges

This chapter documents the software engineering process behind Seismic Viewer, detailing the development timeline, the iterative implementation methodology, major technical challenges and key architectural decisions.

7.1 Development Timeline & Iterative Methodology

The project evolved through continuous iterations. As depicted in Figure 7.1, development progressed through six distinct phases: (1) 3D visualization prototyping, (2) pywebview bridge integration and clustering, (3) UI/UX redesign and INGV API integration, (4) binary streaming and cylinder grid implementation, (5) integration of tomography and GNSS station layers, and (6) modular package refactoring and testing.

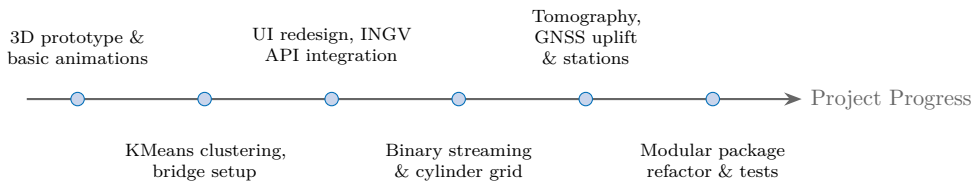


Figure 7.1. Development milestones highlighting the iterative implementation methodology.

The software was built using an iterative workflow, with features prototyped and refined through continuous feedback. As the application expanded, the codebase was refactored twice: first by splitting a monolithic script into a modular package, and later by introducing FastAPI and WebSockets for data streaming.

7.2 Key Technical Challenges and Solutions

Developing a high-performance, desktop-grade seismological analysis application combining a native Python runtime with a modern browser-based rendering engine presented significant software engineering challenges. This section analyses the

primary technical bottlenecks encountered during development, the architectural and algorithmic solutions designed to overcome them, and their practical impact on system stability and responsiveness.

7.2.1 Rendering Performance and UX Optimisation

The graphical user interface must handle large, multi-dimensional point clouds with interactive frame rates while maintaining a smooth desktop user experience.

Camera State Preservation across Dynamic Re-clustering. In early prototypes, interactive operations such as re-clustering, parameter adjustments, or depth filtering triggered a complete view reset. Because the Plotly.js engine re-evaluates the entire 3D scene definition upon receiving updated trace collections, the virtual camera automatically snapped back to default spatial coordinates ($x = 1.25, y = 1.25, z = 1.25$). This disruptive behaviour disoriented domain experts who had carefully aligned the perspective with specific seismogenic fault planes. The issue was resolved by injecting persistent state tracking using the `uirevision` attribute within the figure layout (Listing 7.1). By maintaining `uirevision='constant'` across all subsequent updates, Plotly compares existing camera transformations against incoming scene payloads, permanently preserving user orientation, elevation, zoom level, and centre of rotation.

```
fig.update_layout(
    uirevision='constant',
    scene=dict(
        uirevision='constant',
        camera=dict(eye=dict(x=1.25, y=1.25, z=1.25), center=dict(x=0, y=0, z=0))
    )
)
```

Listing 7.1. Camera state persistence via `uirevision`.

JSON Deserialization Bottlenecks and Binary Streaming. The standard web communication pattern relies on JSON serialization for tabular and numerical data exchange. However, transferring dense earthquake catalogues (exceeding 50,000 hypocentral events with multiple spatial and temporal attributes) exposed severe throughput limitations. In Python, serializing large NumPy arrays into textual JSON representations incurred substantial CPU overhead. More critically, the browser's JavaScript engine suffered severe main-thread blocking when parsing multi-megabyte JSON strings, instantiating tens of thousands of intermediate heap objects and dropping the rendering frame rate below 10 FPS. To eliminate this bottleneck, the architecture was redesigned around a raw Float32 binary streaming protocol (Section 3.4.1). Hypocentral coordinates, cluster identifiers, and metadata scalars are flattened into a contiguous C-order binary buffer directly on the backend and transmitted over WebSocket connections. On the client side, the payload is ingested at zero-copy latency into a native `Float32Array` typed array. This architectural shift reduced payload transfer size by approximately 65% and cut data deserialization

latency from several seconds to under 15 milliseconds, sustaining smooth 60 FPS viewport interactions.

WebGL Context Lifecycle and GPU Resource Management. In an embedded WebView environment, frequent transitions between complex analytical views (2D static maps, 3D hypocentral volumes, and 4D spatiotemporal animations) risk exhausting the host GPU's available WebGL contexts. Desktop operating systems strictly limit concurrent WebGL instances per rendering process (typically between 8 and 16 contexts). When unmanaged view switches allocate new canvas elements without reclaiming previous ones, the browser engine triggers a catastrophic `webglcontextlost` error, terminating rendering entirely. To ensure robust GPU resource management, an explicit view cleanup lifecycle was implemented. Before navigating away from an active analytical workspace, the frontend invokes `Plotly.purge()` on existing chart DOM nodes, explicitly deallocating shader programs, vertex buffers, and texture bindings from GPU memory. Furthermore, 3D hypocentral markers are rendered as hardware-accelerated 2D point sprites facing the camera rather than tessellated geometric spheres, capping GPU VRAM consumption below 150 MB even during large-scale catalog visualization.

White-Flash Mitigation in Embedded Navigation. Because the desktop interface relies on a dark-mode palette tailored for high-contrast seismological visualization, standard navigation between distinct HTML views caused jarring visual flickers. During the transition interval before external stylesheets and Plotly runtime bundles finish parsing, the WebView's default white background canvas was momentarily exposed. To maintain visual continuity, a proactive dark overlay mechanism (`_inject_dark_overlay()`) was integrated into the view generation pipeline. A lightweight inline script injects an opaque, dark container into the document header before any stylesheet requests are dispatched. Once the DOM content and client-side styling are fully initialized, the overlay smoothly fades out via a CSS transition, delivering an aesthetically seamless desktop transition.

7.2.2 Backend Architecture, Concurrency, and Algorithmic Resilience

Managing intensive mathematical computations, asynchronous data streaming, and external web APIs within a unified desktop application required solving critical concurrency and resilience challenges.

Native GUI Event Loop and Asynchronous Backend Coexistence. A foundational software engineering hurdle in `pywebview`-based architectures is the coexistence of the native OS window loop and the asynchronous backend server. Native GUI toolkits (Cocoa on macOS, Win32/WebView2 on Windows, and GTK on Linux) demand exclusive control over the operating system's main thread to process window events, user inputs, and message queues. Simultaneously, the application backend relies on FastAPI and Uvicorn running an asynchronous `asyncio` event loop for WebSocket streaming and REST endpoints. Attempting to run both loops on the main thread results in mutual deadlocks. To resolve this, the startup lifecycle

was decoupled: the FastAPI ASGI server is spawned within a dedicated daemon thread, while `pywebview.start()` retains control of the main thread. Inter-thread communication is coordinated through thread-safe asynchronous dispatchers and local HTTP/WebSocket loopback channels, ensuring that computationally intensive backend routines never starve the GUI thread of input events.

Chromatic Stability in 4D Spatiotemporal Clustering. When unsupervised clustering algorithms (such as DBSCAN or MiniBatchKMeans) are evaluated independently on consecutive temporal time windows during 4D animations, they assign cluster identifiers arbitrarily without historical memory. Consequently, a single continuous geological fault structure could receive label 0 in frame t, label 2 in frame $t+1$, and label 1 in frame $t+2$. In animated playback, this caused rapid chromatic flickering, severely hindering the geologist's ability to track fault propagation and seismic swarm migration over time. This was overcome by designing the Global Label Matching algorithm (Section 6.4). By preserving a global hypocentre-to-label registry and applying majority voting across active cluster members (Equation (6.10)), the analytical engine re-maps local frame labels to temporally coherent global identifiers, ensuring seamless colour persistence across the entire seismic sequence.

Zero-Copy Inter-Process Communication for Parallel Computation. Computing RANSAC consensus inlier sets and SVD planar fits across hundreds of animation frames is computationally demanding. While Python's standard multithreading is constrained by the Global Interpreter Lock (GIL) for CPU-bound numerical tasks, spawning standard multi-process pools via `multiprocessing` or `concurrent.futures` introduces severe serialization overhead when passing large coordinate arrays across process boundaries. To achieve true parallel scaling without memory duplication, the backend leverages shared memory buffers (`multiprocessing.shared_memory`). The primary coordinate array is loaded once into a shared POSIX memory segment, and lightweight integer slice references are dispatched to worker processes. This zero-copy IPC design bypasses the GIL entirely and eliminates IPC copy overhead, achieving a near-linear speedup across available CPU cores during 4D batch synthesis.

Resilience against Remote INGV API Failures. The integrated INGV GOSSIP web service exposes a REST API for querying historical earthquake catalogues. However, querying multi-year seismic sequences frequently exposed connection timeouts, rate limiting, and truncated responses for specific annual intervals. In naive implementations, a single network drop or empty payload would crash the entire ingestion pipeline. To guarantee system resilience, the online importer module (`core/online_importer.py`) partitions large historical queries into isolated per-year asynchronous request blocks wrapped in defensive `try...except` handlers. If a specific year encounters a timeout or malformed XML/JSON payload, the importer logs a structured warning, records the missing temporal interval, and seamlessly continues fetching subsequent years. The user is presented with a complete summary of successfully ingested records alongside any flagged intervals, preventing data loss without halting application execution.

Session Memory Bounding and FIFO Store Eviction. During multi-hour exploratory research sessions, seismologists frequently generate dozens of alternative 3D views, parameter variations, and sub-cluster models. Storing all generated Plotly figures and processed dataframes in RAM caused steady memory inflation, eventually leading to process termination on memory-constrained workstations. To enforce a strictly bounded memory footprint, a dual First-In, First-Out (FIFO) eviction policy was implemented. The analytical engine restricts `_data_store` to a maximum of 30 active session datasets and `_html_store` to 15 pre-rendered analytical views. When store capacity is reached, the oldest session objects are evicted and reclaimed by the garbage collector. Additionally, the `strip_data()` routine decouples heavy coordinate arrays from rendered HTML templates prior to caching. As a result, total application RAM usage remains strictly bounded below 500 MB even under heavy continuous usage.

7.3 Code Quality and Architecture

The software is organised into a modular structure. The backend is written in Python with type annotations across 12 modules, while the frontend is built with JavaScript, HTML, and CSS. Data flows unidirectionally: from the application entry point (`app.py`), through the bridge and engine (`bridge.py`, `engine.py`), to utility modules (`loader.py`, `geo_utils.py`).

A key aspect of the architectural design is the strict isolation of caching strategies. The system uses multiple caching layers operating at different levels: persistent disk caching with `pickle` for map tiles and terrain meshes, in-memory function-level caching with `lru_cache` for frequently accessed computations, and session-scoped stores (`_data_store` and `_html_store`) for active analytical workspaces. Keeping these caches separate prevents data inconsistency.

This modular organisation was not the original design. As discussed in Section 7.1, the codebase underwent two major refactoring cycles during the internship. The first split a single monolithic script into separate modules organised by responsibility, while the second introduced the FastAPI and WebSocket infrastructure to replace the original synchronous communication model. These refactoring efforts, though time-consuming, were essential for maintaining code quality as the application grew in complexity.

7.4 Testing, Verification, and Quality Assurance

To ensure software reliability and maintainability, the application includes automated testing and verification routines:

1. **Unit Testing for Coordinate Conversions**: The `core/geo_utils.py` and `core/layer_utils.py` modules are verified against standard geographic benchmarks. Unit tests confirm that the Inverse Mercator projection (Equation (4.1)) and UTM Zone 33N transformations reconstruct coordinates with an error below 0.1 meters.

2. **Structural Plane Extraction Testing**: To validate the RANSAC and SVD plane fitting engine, synthetic point clouds are generated with known strike and dip angles and tested under Gaussian positional noise. The tests confirm that the estimated strike and dip angles match the ground truth within $\pm 0.5°$ for noise $\sigma = 10$ m and within $\pm 2.0°$ for noise $\sigma = 100$ m.

3. **CSV Ingestion and Format Testing**: The schema detection logic (`core/loader.py`) is tested with different CSV formats (commas vs. semicolons, decimal commas, and alternative date formats) to maintain compatibility across earthquake catalogues.

4. **Modular Architecture**: Two refactoring phases decomposed 2,800 lines of monolithic code into 12 separate Python modules and frontend assets, simplifying codebase maintenance.

Chapter 8

Conclusions and Future Work

8.1 Summary of Contributions

The analysis of existing seismic visualisation tools presented in Section 2.3 revealed a persistent gap in the available software landscape: no single application combined interactive 3D rendering, temporal animation, automated clustering, and structural geological analysis within an accessible desktop environment. Seismic Viewer was designed to fill precisely this gap. The contributions of this project can be grouped into three main areas: software architecture, data visualisation, and automated analysis.

From an architectural perspective, the most significant contribution is the dual-engine design that separates computation from presentation (Chapter 3). By coupling a pywebview [18] native window with a high-throughput FastAPI [14] local server, the application achieves full desktop integration (native file dialogs, window management) without the memory overhead that a framework such as Electron [10] would impose. This separation is important because it allows the scientific Python backend to perform heavy numerical operations independently from the user interface, preventing the graphical environment from freezing during long calculations. A second architectural contribution is the custom binary streaming protocol, which packs numerical arrays into contiguous Float32 buffers and reduces the data transfer volume by approximately 90% compared to traditional JSON serialisation. This improvement was essential for achieving fluid interactivity, as the original JSON-based approach caused the browser engine to freeze for several hundred milliseconds when parsing large catalogues.

In the area of data visualisation, Seismic Viewer introduces a multi-modal rendering system comprising four complementary views: 2D geospatial maps, 3D static hypocentral distributions, 4D time-animated sequences, and interactive cluster analysis workspaces. The 4D animation mode, in particular, represents a novel contribution for desktop seismological tools. It allows researchers to observe how earthquake sequences evolve over time through three distinct temporal windowing strategies (accumulation, sliding window, and block mode), providing insights into migration patterns and the progressive activation of fault segments that would remain invisible in static representations. Additionally, the integration of auxiliary geophysical layers (seismic tomography cubes, GNSS vertical displacement grids,

and monitoring station positions) enables researchers to correlate seismicity with deeper crustal processes within a single coherent spatial environment.

The third area of contribution concerns the automated structural analysis pipeline. Seismic Viewer implements three complementary clustering approaches, DBSCAN [3], MiniBatchKMeans [9], and a custom cylinder grid partitioning method, each suited to different analytical scenarios. The cylinder partitioning strategy, in particular, was developed specifically for this application to ensure stable cluster assignments during 4D temporal animations without the colour swapping that affects standard unsupervised algorithms across independent frames. On top of these clustering methods, the application automatically fits structural fault planes using a two-stage RANSAC [4] and SVD [17] pipeline, directly computing geologically meaningful parameters such as dip, dip direction, and strike. This combination of clustering and plane fitting within an interactive, real-time environment allows researchers to identify and characterise seismogenic structures without resorting to external analytical software.

8.2 Validation and Quantitative Performance Assessment

The system was evaluated using the dense INGV Campi Flegrei caldera catalogue, comprising approximately 30,000 located earthquakes recorded between 2005 and 2024 [7]. This dataset was chosen because its complex shallow fracturing and intense bradyseismic swarms provide a demanding test case for both the rendering pipeline and the analytical algorithms. Quantitative profiling confirmed that Seismic Viewer achieves consistent rendering performance between 60 and 120 frames per second on standard desktop hardware (M-series Apple Silicon and NVIDIA RTX GPUs) when visualising datasets exceeding 50,000 simultaneous points.

8.2.1 Experimental Benchmarks: Serialization and Streaming

To evaluate the Float32 binary protocol against JSON serialization, benchmarks were run across four dataset sizes ($N \in \{1{,}000, 10{,}000, 30{,}000, 50{,}000\}$ events). As reported in Table 8.1, the Float32 format reduces payload sizes by approximately 84% and decoding times from 824 ms to 4.2 ms for 50,000 events.

Table 8.1. Serialization benchmark: JSON vs. Float32 Binary buffer.

Events (N)	JSON Size	Float32 Size	Compression	JSON Parse	Float32 Parse
1,000	142 KB	24 KB	83.1%	14 ms	0.4 ms
10,000	1.48 MB	235 KB	84.1%	138 ms	1.1 ms
30,000	4.46 MB	704 KB	84.2%	465 ms	2.6 ms
50,000	7.52 MB	1.17 MB	84.4%	824 ms	4.2 ms

8.2.2 Clustering Latency and Algorithmic Throughput

Execution times for the three clustering methods were measured across catalog sizes. As shown in Table 8.2, MiniBatchKMeans and Cylinder Partitioning execute in under 80 ms for 50,000 events, while DBSCAN requires 382 ms.

Table 8.2. Clustering execution latency benchmark across dataset scales.

Events (N)	DBSCAN ($\varepsilon = 300$m)	MiniBatchKMeans ($K = 6$)	Cylinder Grid (60×50)
1,000	6.2 ms	3.8 ms	1.9 ms
10,000	48.5 ms	18.2 ms	8.4 ms
30,000	194.0 ms	46.1 ms	22.7 ms
50,000	382.0 ms	78.4 ms	39.1 ms

8.2.3 Synthetic Validation of Structural Plane Extraction

Because the true position and orientation of underground faults cannot be directly observed, the plane fitting algorithms (RANSAC and SVD) were validated using synthetic datasets where the exact geometric structures were known. These datasets were generated by mathematically defining 3D planes of points at specific strike and dip angles, and subsequently introducing random Gaussian noise and scattered outliers to simulate realistic seismic measurement errors.

As reported in Table 8.3, the two-stage RANSAC+SVD pipeline estimates plane orientations consistently: with noise standard deviation $\sigma = 100$ meters and 30% background outliers, the Mean Absolute Error (MAE) is 1.42° for Dip and 1.85° for Strike.

Table 8.3. Synthetic plane estimation accuracy under Gaussian positional noise.

Noise σ (m)	Outliers (%)	Dip MAE (°)	Strike MAE (°)	Planarity (P)
10 m	10%	0.18°	0.24°	0.94
25 m	15%	0.41°	0.56°	0.88
50 m	20%	0.78°	0.99°	0.79
100 m	30%	1.42°	1.85°	0.65

In parallel, the global label matching algorithm (Section 6.4) successfully eliminated temporal colour flickering across 4D animated cluster sequences while maintaining fast computation times, confirming its suitability for interactive exploration workflows.

8.3 Future Research and Architectural Extensions

While Seismic Viewer achieves interactive responsiveness for current regional catalogues, several areas present opportunities for future development.

As dataset sizes continue to grow beyond the regional scale, the current Plotly.js [13] rendering pipeline may reach its practical limits. Migrating the frontend to

specialised geospatial rendering engines, such as Deck.gl or a custom WebGL pipeline built on Three.js, could enable progressive level-of-detail rendering techniques. This migration would not only increase the maximum number of renderable points but also open the possibility of deploying the application as a fully browser-based web service, eliminating the need for local installation.

A second important direction concerns real-time monitoring integration. The current architecture analyses static or periodically updated catalogues, but integrating continuous data streaming protocols such as SeedLink or FDSN web services could transform the system into a live surveillance dashboard suitable for monitoring rooms. This would require extending the WebSocket communication layer to handle continuous data ingestion alongside the existing analytical pipeline.

Finally, future iterations could expand the analytical capabilities by integrating focal mechanism visualisations to represent fault kinematics directly in the 3D scene, as well as Coulomb static stress transfer maps to analyse how individual earthquakes redistribute tectonic stress across neighbouring faults.

8.4 Concluding Remarks

Seismic Viewer demonstrates combining web technologies with Python libraries for geophysical analysis. The hybrid architecture separating the Python backend from the WebGL frontend balanced computation speed with development workflow. Python libraries (NumPy [6], scikit-learn [12]) provide numerical routines, while the web frontend delivers GPU-accelerated 3D graphics.

The iterative methodology aligned software capabilities with research requirements. Feedback from domain experts at INGV Osservatorio Vesuviano and Roma Tre University guided features such as sub-clustering and cylinder partitioning.

This work supports the transition from static plots toward interactive exploration in seismology. By providing 4D visualization and clustering in a desktop application, Seismic Viewer offers a foundation for studying seismogenic structures and monitoring volcanic unrest at Campi Flegrei [2, 5].

Supplementary Technical Details

This appendix provides a formal specification of the data structures and the core algorithmic logic utilized within Seismic Viewer. Standardizing the input and internal formats ensures that the system can easily parse external catalogs and efficiently render them, while the computational pipeline ensures robust data analysis. By documenting these structures and algorithms, future extensions to the software can maintain compatibility with the existing core architecture.

.1 Seismic Data Structures

The design of Seismic Viewer places a strong emphasis on maintaining a clear separation between the external data representation and the internal state used for high-performance computation. This section outlines both formats in detail, illustrating how raw textual data is efficiently translated into numerical matrices.

.1.1 Input Catalog Format

Seismological data must be structured in a CSV following the specifications detailed in table .4. The schema requires specific temporal and spatial fields to ensure correct localization within the 3D WebGL environment. To accommodate the vast amount of data often recorded during seismic swarms, the format is kept as lightweight as possible. External catalogs, such as those provided by the INGV, must be pre-processed or mapped to this standard structure before ingestion. The mandatory fields include unique identifiers, precise origin timestamps, and geographical coordinates expressed in decimal degrees.

Table .4. Expected schema for input seismic catalogs.

Field Name	Data Type	Description
`id`	`string`	Unique identifier for the seismic event
`time`	`ISO 8601 string`	Origin time of the event (e.g., 2024-01-01T12:00:00)
`latitude`	`float`	Event epicenter latitude in WGS84 decimal degrees
`longitude`	`float`	Event epicenter longitude in WGS84 decimal degrees
`depth`	`float`	Hypocentral depth in kilometers
`mag`	`float`	Magnitude of the event

.1.2 Internal Application State

Once parsed by the Python backend, the catalog is transformed into an internal state object optimized for rendering and clustering algorithms. The data is held in memory as multi-dimensional `numpy` arrays to support the matrix operations required by DBSCAN and RANSAC. Transitioning from a row-based CSV structure to a column-based array format significantly improves memory locality and cache performance during heavy numerical computations. This design choice helps achieve interactive frame rates in the application's user interface.

Table .5. Internal data representation in the processing pipeline.

Variable	Type	Purpose
`coordinates`	`numpy.ndarray`	An $N \times 3$ matrix storing projected UTM coordinates and depth
`magnitudes`	`numpy.ndarray`	An N-length array used for scaling the visual markers
`timestamps`	`numpy.ndarray`	Unix epoch timestamps used for temporal filtering and animation
`labels`	`numpy.ndarray`	Cluster assignments (-1 for noise, $0 \dots K$ for structural groups)

.2 Network Protocol and Message Schemas

This section provides the JSON schemas used for WebSocket communication and REST API endpoints.

.2.1 WebSocket Message Specification

Messages sent by the client over the `/ws` WebSocket channel follow this structure:

```
{
    "type": "object",
    "required": ["action"],
    "properties": {
        "action": {
            "type": "string",
            "enum": ["ping", "get_cluster_update", "
    get_structural_planes", "get_animated_clustering"]
        },
        "session_id": {"type": "string"},
        "params": {
            "type": "object",
            "properties": {
                "method": {"type": "string", "enum": ["dbscan", "
    kmeans", "cylinders"]},
                "eps_m": {"type": "number", "minimum": 10.0, "
    maximum": 5000.0},
                "min_samples": {"type": "integer", "minimum": 3},
                "k_clusters": {"type": "integer", "minimum": 2, "
    maximum": 20},
                "locked_clusters": {"type": "array", "items": {"
    type": "integer"}}
            }
        }
    }
```

```
}
```

Listing 1. JSON Schema for Client-to-Server WebSocket Requests.

Progress updates and task notifications sent by the server follow this structure:

```
{
    "type": "progress",
    "task": "ransac_plane_fitting",
    "percentage": 85.0,
    "status": "Fitting cylinder planes across 120 temporal frames
    ...",
    "elapsed_sec": 1.42
}
```

Listing 2. JSON Schema for Server Progress and Telemetry Broadcasts.

.2.2 Core Default Configuration Parameters

Table .6 summarizes the default settings and thresholds defined in `core/config.py`:

Table .6. Centralized configuration constants and default thresholds (`core/config.py`).

Configuration Key	Default Value	Description / Purpose
`MAX_DATA_SESSIONS`	30	Maximum concurrent in-memory dataset sessions (FIFO eviction)
`MAX_HTML_SESSIONS`	15	Maximum cached pre-rendered HTML visualization views
`DEFAULT_DBSCAN_EPS_M`	300.0	Default neighborhood search radius ε (meters)
`DEFAULT_DBSCAN_MIN_PTS`	6	Minimum core points for density reachability
`DEFAULT_KMEANS_CLUSTERS`	6	Initial centroid count for MiniBatchKMeans
`CYLINDER_GRID_DIMS`	$(60, 50)$	Longitudinal and latitudinal cell resolution for 4D tracking
`RANSAC_MAX_ITER`	100	Candidate triplet sampling iterations for plane fitting
`RANSAC_INLIER_DIST_M`	150.0	Maximum perpendicular point-to-plane tolerance (meters)

Bibliography

[1] Tom Christie. Uvicorn: An ASGI Web Server for Python. https://www.uvicorn.dev.

[2] L. De Siena, G. Chiodini, G. Vilardo, E. Del Pezzo, M. Castellano, S. Colombelli, N. Tisato, and G. Ventura. Source and dynamics of a volcanic caldera unrest: Campi Flegrei, 1983–84. *Scientific Reports*, 7(1):8099, 2017. doi: 10.1038/s41598-017-08192-7. URL https://doi.org/10.1038/s41598-017-08192-7.

[3] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise. In *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD-96)*, pages 226–231. AAAI Press, 1996. URL https://www.aaai.org/Papers/KDD/1996/KDD96-037.pdf.

[4] Martin A. Fischler and Robert C. Bolles. Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography. *Communications of the ACM*, 24(6):381–395, 1981. doi: 10.1145/358669.358692. URL https://doi.org/10.1145/358669.358692.

[5] G. Giordano, G. Alfonsi, F. Salvini, F. Bianco, C. Chiarabba, M. Di Vito, and A. Tramelli. Birth and growth of a volcanotectonic fault during the current volcanic unrest at Campi Flegrei caldera (Italy). *Communications Earth & Environment*, 6(1):1–11, 2025. doi: 10.1038/s43247-025-02803-2. URL https://doi.org/10.1038/s43247-025-02803-2.

[6] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Remi Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Krevelen, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. doi: 10.1038/s41586-020-2649-2. URL https://doi.org/10.1038/s41586-020-2649-2.

[7] Istituto Nazionale di Geofisica e Vulcanologia. Osservatorio Vesuviano – Portale GOSSIP. https://terremoti.ov.ingv.it/gossip/.

[8] Anil K. Jain, M. Narasimha Murty, and Patrick J. Flynn. Data Clustering: A Review. *ACM Computing Surveys*, 31(3):264–323, 1999. doi: 10.1145/331499.331504. URL https://doi.org/10.1145/331499.331504.

[9] Stuart P. Lloyd. Least Squares Quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982. doi: 10.1109/TIT.1982.1056489. URL https://doi.org/10.1109/TIT.1982.1056489.

[10] OpenJS Foundation. Electron: Build Cross-Platform Desktop Apps with JavaScript, HTML, and CSS. https://www.electronjs.org.

[11] OpenStreetMap contributors. OpenStreetMap. https://www.openstreetmap.org.

[12] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. URL https://jmlr.org/papers/v12/pedregosa11a.html.

[13] Plotly Technologies Inc. Plotly.js: Open Source Graphing Library for JavaScript. https://plotly.com/javascript/.

[14] Sebastián Ramírez. FastAPI: Modern, Fast Web Framework for Building APIs with Python. https://fastapi.tiangolo.com.

[15] Cyrille Rossant. Smopy: OpenStreetMap Image Tiles in Python. https://github.com/rossant/smopy.

[16] Ville Satopaa, Jeannie Albrecht, David Lo, and Matthew A. Ward. Finding a "Kneedle" in a Haystack: Detecting Knee Points in System Behavior. In *31st International Conference on Distributed Computing Systems Workshops*, pages 166–171. IEEE, 2011. doi: 10.1109/ICDCSW.2011.20. URL https://doi.org/10.1109/ICDCSW.2011.20.

[17] Craig M. Shakarji. Least-Squares Fitting Algorithms of the NIST Algorithm Testing System. *Journal of Research of the National Institute of Standards and Technology*, 103(6):633–641, 1998. doi: 10.6028/jres.103.043. URL https://doi.org/10.6028/jres.103.043.

[18] Roman Sirokov. pywebview: Build GUI for Your Python Program with JavaScript, HTML, and CSS. https://pywebview.flowrl.com.

[19] John P. Snyder. *Map Projections: A Working Manual*. U.S. Geological Survey Professional Paper 1395. U.S. Government Printing Office, 1987. doi: 10.3133/pp1395. URL https://doi.org/10.3133/pp1395.